

Memory-, Latency-, and Energy-Aware Allocation of Quantized Small Language Models on a Single 8 GB GPU

A measured, coupling-aware mixed-integer optimization for an on-device multi-agent legal-assistant pipeline

(Author names withheld for review)

Target hardware: NVIDIA RTX 4070 Laptop GPU (8 GB) • Case study: an on-device legal assistant

Abstract

Deploying a multi-agent language-model pipeline on a single consumer GPU is constrained by a fixed memory budget and by the latency and energy cost of generation. We make three contributions, all grounded entirely in on-device measurement. *First*, we characterize the performance of 75 quantized small-language-model (SLM) configurations—10 open-weight models (0.36–7.2 B parameters) at three `llama.cpp` K-quantization levels and three context lengths—measured on one NVIDIA RTX 4070 Laptop GPU. We report time-to-first-token, decode throughput, peak VRAM, and energy per token, and we distil them into the precision, context, and scale cost curves that govern deployment. *Second*, we pose and solve a mixed-integer linear program (MILP) that, using only these measurements, assigns a configuration to each of the pipeline’s three generation roles so as to maximize distinct-model capacity subject to a load-once memory budget and a serial system-latency budget ε , parametric in the number k of activated specialists. The frontier is memory-bound at ≈ 11.9 B and shifts rightward with k . We report a deliberately strict baseline: for this (sortable) parameter objective a greedy heuristic matches the MILP on 92% of points, so optimization adds little—and we say so. Its value appears once the objective is not sortable: with heterogeneous specialists (distinct legal domains, per-domain model affinities) the MILP beats greedy on 92% of points and on all points when enough specialists are active. Finally, we report the measured-quality campaign across all three roles: quality is *non-monotone* in parameter count for the specialists—the best model is Llama-3.2-1B, beating Mistral-7B at $6\times$ fewer parameters—so the capacity-maximizing allocation is sub-optimal in quality at every latency budget; conversely, routing F1 (dispatcher) and synthesis quality both *rise* with model size. The three roles thus prefer opposite ends of the model-size spectrum. Acting on this, we build a *quality-aware* allocator—the same MILP with measured quality replacing the proxy objective—and trace the measured quality–latency frontier. Given only measured quality it independently recovers the three-regime structure (large dispatcher, small specialist, large synthesiser), beats both fixed-model policies, and at matched latency dominates the parameter-proxy allocator at every budget (mean quality gap 0.13): the parameter proxy and a measured-quality objective induce materially different optima. Finally, we model the router–specialist *coupling* (a specialist is wasted if the router misroutes) as a bilinear term, linearize it exactly with McCormick envelopes, and solve it on measured data: the gated optimum dominates the coupling-blind additive optimum at 42 of 44 budgets and selects a small, high-recall router that global-F1-based selection misses. Finally, we ask which of these modelling choices are load-bearing: a single-family ablation shows that *both* the non-monotone specialist regime and the value of modelling router–specialist coupling are cross-family phenomena—each collapses within a single dense family and re-appears only across families—whereas the small-specialist optimum is robust to the serial-versus-batched execution model. The diversity of the candidate pool, not the latency accounting, is what the conclusions depend on. Throughout, “capacity” is a parameter-count *proxy*, and the heterogeneous affinities of the optimization section are *synthetic* and labelled as such. *Third*, holding the static allocation fixed, we add a dynamic per-query layer that selects which specialists to activate from a calibrated, LLM-free retrieval gate adding no model to the

memory budget: on a held-out test set of 64 queries it raises routing F_1 from 0.33 to 0.50 and cuts latency 9–35% (concurrent vs. sequential) with a gain independent of the underlying allocation, while abstaining on out-of-scope queries; and a single-objective concurrent-batch MILP confirms batched serving dominates sequential serving in quality and feasibility. All data, the MILP, the baselines, and the plotting code are released.

1 Introduction

The practical bottleneck for running language models on laptop-class hardware is memory, not arithmetic: a model and its key–value (KV) cache must fit in a fixed VRAM budget, often 8 GB or less. A *multi-agent* pipeline compounds this, because several specialised agents must co-reside and cooperate on every query. The motivating application here is an on-device assistant for a confidentiality-sensitive domain (legal question answering), composed of a routing *dispatcher*, nine domain *specialists*, and a *synthesiser* that produces the final answer—all of which must share one GPU.

Three questions, one thread. This paper is organized around three questions that build on one another, and we use them as a thread through the results:

- Q1 (Cost).** *Which configurations are feasible, and what do they cost?* We measure all 75 quantized configurations on the GPU (§4–5) and pose an allocation MILP that maximizes a transparent capability *proxy*—distinct-model parameter count—under the memory and latency budgets (§6–7). The honest finding: a greedy sort matches the MILP on 92% of points, so for a sortable proxy the optimization barely earns its keep.
- Q2 (Capability).** *Is the proxy the right objective?* We run a measured-quality campaign across all three roles (§8) and re-optimize on measured quality (§9). The answer is no: quality is non-monotone in size, the roles follow three distinct scaling laws, and the quality-aware optimum is a *mixed* allocation that beats the proxy at every budget.
- Q3 (Coupling).** *Do the roles interact?* A specialist is wasted if the dispatcher misroutes it, so we model the router–specialist coupling as a *bilinear* term, solve it exactly on measured data (§10), and sweep the coupling strength to show *when* optimization is worth it. The gated optimum differs from the additive one, and the value of optimization grows with coupling.
- Q4 (Generality).** *Which modelling choices are load-bearing?* The results rest on two deliberate choices: a *heterogeneous* pool spanning six model families, and a *serial* latency chain. We test both—we restrict the pool to a single dense family (§11.1) and we re-solve the quality-aware allocator under an exact *batched* execution model (§10.5)—and find an asymmetry: the non-monotone specialist regime collapses within one family, whereas the small-specialist optimum is robust to the execution model. Heterogeneity is load-bearing for the *finding*; the latency model is not load-bearing for the *optimum*.
- Q5 (Adaptivity).** *Once the substrate is allocated, can the pipeline adapt per query?* The allocation above is *static*: every query pays for the same activated specialists. We add a *dynamic* layer (§12) that, holding the allocation fixed, decides per query *which* specialist domains to run—making the activation count $k(q)$ endogenous—from a calibrated, LLM-free retrieval gate that loads no model and so adds nothing to the memory budget. On held-out queries it raises routing F_1 from 0.33 to 0.50 and cuts latency 9–35% (concurrent vs. sequential) with a gain *independent* of the underlying allocation, and a single-objective concurrent-batch MILP confirms batched serving dominates sequential serving in quality and feasibility.

The arc is therefore: cost says “optimization is optional,” capability says “the objective was wrong,” coupling says “the roles do not decouple, so a per-role exact allocator is what the problem needs,” generality says “and the result depends on pool diversity, not on the execution accounting,” and adaptivity says “and the allocated substrate can be made query-adaptive at no extra memory cost.” Throughout we separate *cost* (measured) from *capability* (measured quality), and we are explicit about what each step establishes.

2 Related Work

Our problem sits at the intersection of five lines of work: model cascades and cost-aware serving, query-level routing, multi-agent LLM pipelines, reference-free RAG evaluation, and on-device quantized inference. We review each and then state precisely how MAMAP-Edge differs.

Model cascades and cost-aware serving. Cascades route easy queries to small models and defer hard ones to large models, trading latency for quality [2, 3]. The closest work to ours is **Cascadia** [1], which also uses a mixed-integer linear program for LLM serving: at its inner level it selects resource allocations and parallelism strategies, while an outer level co-optimizes the routing policy. Chen et al. [4] unify routing and cascading into a single framework, prove optimality conditions, and—most relevant to us—identify *when* each paradigm helps, showing that the quality of the per-model quality estimator is the deciding factor. A recent survey [5] catalogues this fast-growing area. These systems share our concern with the cost–quality frontier and our use of integer programming, but they all operate *per query* on a cascade or routing decision over interchangeable general-purpose models. We instead make a *static* allocation of *fixed* models to *fixed heterogeneous roles* in a parallel multi-agent pipeline, under a hard single-GPU memory budget—a different decision variable and a different constraint regime.

Query-level routing. **RouteLLM** [6] learns, from human-preference data, to route each query to a strong or weak model, cutting cost by more than 2× without quality loss and transferring across model pairs. Routing answers “which single model for *this* query”; our dispatcher does route queries to specialist domains, but the paper’s optimization question is orthogonal: “which model should permanently *serve* each role,” given that all roles must co-reside on one GPU. Routing and our allocation are complementary—a deployed MAMAP-Edge pipeline still routes queries at run time; we optimize the fixed substrate it routes over.

Multi-agent LLM pipelines. Multi-agent systems decompose a task across collaborating LLMs. **Mixture-of-Agents** [7] layers “proposer” models whose outputs an “aggregator” synthesizes—structurally identical to our specialists-plus-synthesiser stage—and shows open models can collaborate to match a larger one. Surveys [8] taxonomize collaboration structures (centralized, peer-to-peer), and recent work questions when mixing *different* models actually helps. This literature studies the *quality* benefit of collaboration but almost never the *deployment* question of fitting the agents onto constrained hardware, nor the *coupling* between an upstream router’s accuracy and a downstream agent’s usefulness, which we model explicitly.

Reference-free RAG evaluation. **RAGAS** [9] introduced reference-free metrics—faithfulness, answer relevance, and context relevance—for RAG pipelines, validated against human judgement and now widely used. Our quality campaign adopts the RAGAS metric family (faithfulness, answer relevancy, context precision/recall) but computes the generation metrics with embedding similarities under an *oracle-retrieval* protocol, isolating generation quality from retrieval quality (§3.5); we use these scores not to evaluate a single system but as the *objective* of

an allocation optimizer.

On-device and quantized small models. A growing body of work studies running quantized models at the edge with `llama.cpp`/GGUF and related runtimes [11, 12]. Notably, **SLMQuant** [10] shows that small models have *different* quantization sensitivity from large ones, so LLM-tuned recipes transfer poorly—direct evidence that SLM behaviour must be characterized, not extrapolated, which motivates our exhaustive 75-configuration measurement. Unified quantization evaluations on single models [13] report per-format quality/throughput trade-offs; we extend this from a single model to a pool of 11 models across a multi-role pipeline, and—crucially—tie the measured quality back into an allocation decision.

What is new here. To our knowledge, no prior work combines these threads as we do: (i) a *static, role-aware* allocation of fixed quantized SLMs to the three generation roles of a multi-agent pipeline, under a hard load-once memory budget and a serial latency budget on a single consumer GPU; (ii) a *measured-quality* objective across all roles, showing that a parameter-count proxy and measured quality induce materially different optima, and that quality is non-monotone in model size with three distinct per-role scaling laws; and (iii) an explicit *bilinear* model of router–specialist coupling, linearized exactly and solved on measured data, with a coupling sweep that quantifies *when* optimization beats a greedy heuristic. Cascadia [1] and cascade routing [4] are the nearest neighbours in method (integer programming for serving), but they optimize per-query cascade/routing over interchangeable models, whereas we optimize a fixed multi-agent substrate with heterogeneous roles and measured, coupled per-role quality.

3 Experimental Setting

This section specifies, precisely, the system under study: its agents and their roles, the model pool, the evaluation data and legal corpora, the inference stack, and the quality metrics. All artifacts are released.

3.1 Agent architecture and roles

The application is an on-device assistant for **Italian family law**, organized as an eleven-agent pipeline with three role types (Table 1). A single **dispatcher** reads the incoming query and routes it to a subset of the nine domain **specialists**; the activated specialists answer in parallel using retrieved legal context; and a single **synthesiser** composes the activated specialists’ answers into the final user-facing response. The dispatcher is on the critical path but generates only a short routing decision; the specialists and the synthesiser generate full answers. Each agent declares a minimum context length $c_{\min}$ it requires (Table 1), which constrains which configurations may serve it. The per-query execution order—dispatcher, then the activated specialists run serially on their shared instance (stage cost $k L_{\text{spec}}$), then synthesiser—defines the system latency chain (§6); the batched alternative, in which the stage cost is a single specialist generation, is reported as the opposite extreme in §10.5.

Table 1: The eleven agents (from `agents.yaml`). The nine specialists map one-to-one to Italian family-law domains. $c_{\min}$ is the minimum context the agent requires.

Role	Agent (domain)	$c_{\min}$
dispatcher	routing / classification	2048
specialist	testate succession (<i>succ. testamentaria</i>)	4096
specialist	intestate succession (<i>succ. legittima</i>)	4096
specialist	consensual separation (<i>sep. consensuale</i>)	4096
specialist	contested separation (<i>sep. contenziosa</i>)	8192
specialist	assisted negotiation & mediation	4096
specialist	minor protection (<i>tutela minori</i>)	8192
specialist	community of property (<i>patr. comunione</i>)	4096
specialist	separation of property (<i>patr. separazione</i>)	4096
specialist	non-contentious jurisdiction (<i>vol. giurisdizione</i>)	4096
synthesiser	final-answer composition	8192

3.2 Model pool

Any agent can be served by any of **10 open-weight instruction-tuned models** spanning 0.36–7.2 B parameters (Table 2), each available in 2–3 `llama.cpp` K-quantizations and three context lengths (2048/4096/8192). The full grid yields **75 candidate configurations** (model $\times$ quantization $\times$ context). The pool deliberately mixes six dense decoder-only families (Qwen2.5, Llama-3.2, Gemma-2, Phi-3.5, SmolLM2, Mistral).

Table 2: The 10-model pool (from `catalog_zoo.yaml`). Each model is built at the listed quantizations and three contexts (2048/4096/8192), giving 75 configurations in total.

Key	HuggingFace id	Params	Quantizations
smollm2-360m	SmolLM2-360M-Instruct	0.36	Q3/Q5/Q8
qwen2.5-0.5b	Qwen2.5-0.5B-Instruct	0.5	Q3/Q5/Q8
llama3.2-1b	Llama-3.2-1B-Instruct	1.2	Q5/Q8
qwen2.5-1.5b	Qwen2.5-1.5B-Instruct	1.5	Q3/Q5/Q8
smollm2-1.7b	SmolLM2-1.7B-Instruct	1.7	Q3/Q5/Q8
gemma2-2b	gemma-2-2b-it	2.6	Q5/Q8
qwen2.5-3b	Qwen2.5-3B-Instruct	3.0	Q3/Q5/Q8
llama3.2-3b	Llama-3.2-3B-Instruct	3.2	Q5/Q8
phi3.5-mini	Phi-3.5-mini-instruct	3.8	Q3/Q5
mistral-7b	Mistral-7B-Instruct-v0.3	7.2	Q3/Q5

3.3 Evaluation data and legal corpora

Query set. Evaluation uses **25 hand-curated Italian family-law queries** (`calib_clean_with_gold_text.y`) each annotated with: the expected specialist set (gold routing), a difficulty label (simple/medium/complex: 5/8/12), a risk level (low/medium/high: 8/8/9), a category, and a **gold answer with gold supporting authorities**—both statute IDs/passages and case IDs/passages. Queries activate 2.68 specialists on average (median 2, max 6), so most require multi-specialist synthesis. We note plainly that 25 queries is a small evaluation set (§8, § Limitations).

Retrieval corpora. Specialists answer with retrieval-augmented generation over two curated Italian legal corpora: a **statute corpus** of 63 articles (`LawCorpus.IT`; 60 Codice Civile, 2 Codice di Procedura Civile, and Art. 5 D.Lgs. 28/2010 on mediation), and a **case-law corpus** of 61 items (`CaseCorpus.IT`; 57 court decisions identified by ECLI, 3 mediation records, 1 notarial

act). The gold authorities annotated on each query are drawn from these corpora, enabling the retrieval and grounding metrics below.

3.4 Inference stack and measurement

All models run fully GPU-offloaded via `llama.cpp` on a single **NVIDIA RTX 4070 Laptop GPU** (8 GB, ≈ 6.99 GB usable) under the Windows WDDM driver, with greedy decoding. For every configuration we measure peak VRAM, time-to-first-token, throughput, and energy per token (`perf_table.parquet`; medians over repeated runs), as detailed in §4. Latencies for an N -token answer are derived as $\text{TTFT} + N/\text{throughput}$, with role-specific N (15 routing tokens for the dispatcher, 384 for specialists and the synthesiser).

3.5 Retrieval: corpus indexing, chunking, and FAISS

The specialists and synthesiser are retrieval-augmented. We describe the indexing pipeline (`build_faiss.py`, `scoring/{corpus, embeddings, retrieval}.py`) exactly as implemented.

Unit of indexing (“chunking”). The corpora are pre-segmented at the *legal-unit* level rather than by fixed-size windows: each statute article (e.g. `IT_CC_art_602`) and each case/mediation/notarial passage (e.g. `ECLI:IT:TRGO:2018:431`) is one atomic document with its own id and text. The index is therefore built over 63 statute units and 61 case units (§3); there is no sub-article windowing or overlap, because Italian civil-code articles are already short, self-contained legal norms and splitting them would fragment a single rule across chunks. Each (`id`, `text`) pair maps one-to-one to a vector and a manifest entry.

Embeddings. Documents and queries are embedded with **BAAI/bge-m3** (multilingual, including Italian; 1024-dimensional) via `sentence-transformers`. All vectors are **L2-normalized**, so inner product equals cosine similarity. A dependency-free hashing embedder (also 1024-d, unigram+bigram) is provided solely to validate the pipeline offline; all reported results use `bge-m3`.

FAISS index. For each corpus we build a `IndexFlatIP` (exact, brute-force inner-product) index and persist it alongside a JSON *manifest* listing the document ids in index-add order—FAISS stores only vectors, so the manifest is what maps a hit back to its id and text. We keep two separate indexes (statutes and case law) mirroring the gold-set split. Because the corpora are small ($\mathcal{O}(10^2)$ vectors), an exact flat index is both feasible and preferable to an approximate one (IVF/HNSW): retrieval latency is negligible and recall is not sacrificed to quantization. At query time the embedded query searches both indexes for the top- k , results are merged, deduplicated by id, and the top- k passage texts are returned as RAG context (the live retriever uses $k=5$).

Oracle-retrieval evaluation (important). For the quality campaign in this paper, the generation metrics are computed under *oracle retrieval*: each query carries gold supporting-authority ids, and their texts are supplied directly as the context (`CorpusStore.contexts_for`). This isolates *generation* quality from *retrieval* quality—we measure how well a model uses the correct context, not whether FAISS surfaces it. A direct consequence is that context-precision/recall against the gold ids are trivially high by construction and are *not* informative about the live retriever in these runs; the separate retriever diagnostic (§8.10) reports the live FAISS retrieval quality. We state this explicitly because it bounds what the quality numbers do and do not establish.

3.6 Quality metrics

Quality is scored **judge-free**: routing by set comparison, generation by embedding similarity, with no LLM judge in the reported pipeline (a RAGAS-with-judge path exists but is optional). All sub-scores lie in $[0, 1]$. Let $\text{emb}(\cdot)$ be the L2-normalized bge-m3 embedding and $\cos(u, v) = u^\top v$.

Dispatcher — routing F1. Routing is multi-label set prediction. With predicted specialist set P and gold set G ,

$$\text{prec} = \frac{|P \cap G|}{|P|}, \quad \text{rec} = \frac{|P \cap G|}{|G|}, \quad F_1 = \frac{2 \text{prec} \cdot \text{rec}}{\text{prec} + \text{rec}}, \quad (1)$$

and the dispatcher’s quality is $Q_{\text{disp}} = F_1$.

Specialists/synthesiser — generation metrics. Given the produced answer a , the gold answer g , and retrieved contexts $\{c_1, \dots, c_m\}$:

$$\text{CORRECTNESS} = \max(0, \cos(\text{emb}(a), \text{emb}(g))), \quad (2)$$

$$\text{FAITHFULNESS} = \max(0, \max_{1 \leq i \leq m} \cos(\text{emb}(a), \text{emb}(c_i))), \quad (3)$$

$$\text{ANS. RELEVANCY} = \cos(\text{emb}(a), \text{emb}(q)), \quad (4)$$

i.e. correctness is semantic agreement with the gold answer, faithfulness is the answer’s maximal support by any single context passage, and relevancy is answer–question alignment. Context precision/recall are computed by id-set comparison against the gold authorities and, under oracle retrieval (§3.5), are reported only as diagnostics.

Aggregate. The scalar quality used everywhere (and as the quality-aware MILP objective, §9) is

$$Q = w_c \cdot \text{CORRECTNESS} + (1 - w_c) \cdot \frac{1}{2} (\text{FAITHFULNESS} + \text{ANS. RELEVANCY}), \quad w_c = 0.5, \quad (5)$$

with NaN sub-scores dropped and weights renormalized. **Context precision/recall are deliberately excluded from Q :** they characterize the retriever, not the candidate model, and would otherwise reward a model for the retriever’s behaviour. For the **synthesiser** we set $Q = \text{CORRECTNESS}$ (grounding is still measured and stored), because the synthesiser’s job is a correct final answer; §8 shows this choice and the aggregate above coincide closely in practice (correctness dominates the aggregate).

The campaign scores each (agent, configuration, query) triple and aggregates to per-(agent, configuration) scores (**quality_scorecard.csv**). One evaluation convention is load-bearing and stated here: the synthesiser is scored on the activated specialists’ *reference* answers (their largest-context configuration), so its score is independent of the allocation and keeps the optimizer linear (§6).

4 Measurement Methodology

Configuration space. We evaluate 10 instruction-tuned open-weight models spanning 0.36–7.2 B parameters (SmolLM2-360M/1.7B, Qwen2.5-0.5B/1.5B/3B, Llama-3.2-1B/3B, Gemma-2-2B, Phi-3.5-mini, and Mistral-7B), each in GGUF format at three `llama.cpp` K-quantizations (Q3_K_M, Q5_K_M, Q8_0) and three context lengths (2048, 4096, 8192). Not every model is built at all three quantizations (Llama-3.2, Gemma-2, Phi-3.5, and Mistral-7B omit some), so the catalogue yields 75 **configurations** (model $\times$ quantization $\times$ context), all of which we measure.

Measured quantities. For each configuration, fully GPU-offloaded, we record time-to-first-token (TTFT), decode throughput (tok/s), peak VRAM (NVML, warm minus baseline), and energy per token (board power via `nvidia-smi`, minus idle, per token), each the median over repeated runs. Decoding is greedy, with identical GGUF artefacts, so measurements are deterministic.

Derived latency and energy. Because end-to-end latency depends on output length, we separate the device-fixed rate from the token count. For an answer of T tokens,

$$L(T) = \text{TTFT} + \frac{T}{\text{throughput}}, \quad E(T) = T \cdot e_{\text{tok}}, \quad (6)$$

and we report L, E at $T = 384$ throughout. Equation (6) assumes throughput is independent of T , so L is mildly optimistic for very long outputs.

Hardware and budget. All numbers are from one NVIDIA RTX 4070 Laptop GPU (8 GB GDDR6); the auto-detected usable budget at test time is $M \approx 6.99$ GB. The device is the unit of measurement.

5 Performance Characterization

Feasibility and throughput. All 75 measured configurations fit within M (Fig. 1): the catalogue’s quantization choices already exclude the configurations that would not fit (notably 8-bit variants of the largest models), so feasibility is governed almost entirely by precision—every retained 3-, 5-, and 8-bit configuration fits the 6.99 GB budget. Decode throughput falls as an approximate power law $\propto N^{-0.6}$ in parameter count N , from 247 tok/s (0.36 B) to 39 tok/s (the 7.2 B Mistral), with heavier quantization consistently slower at fixed size (Fig. 2).

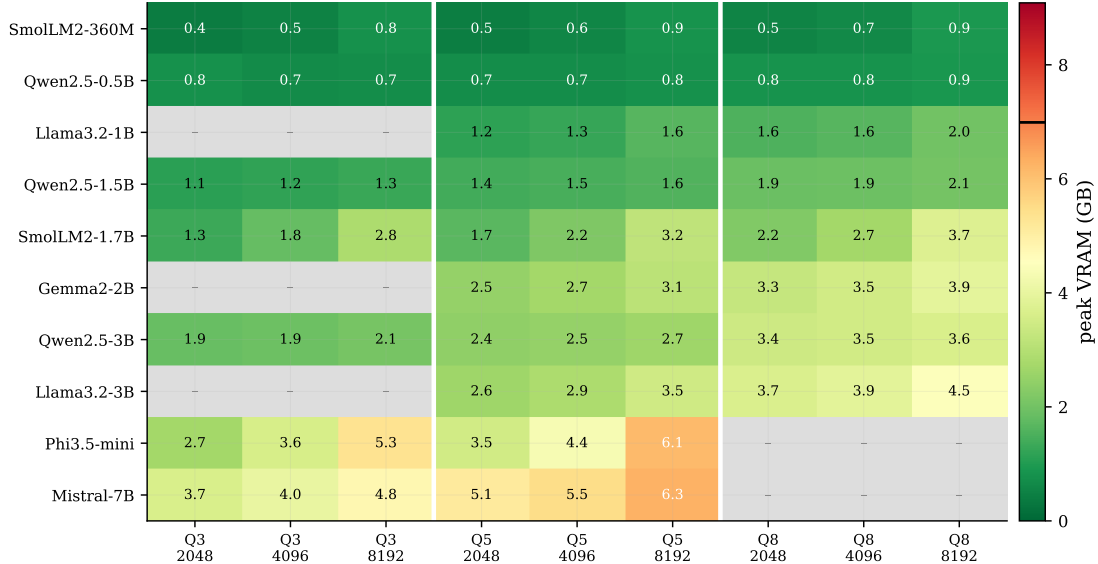


Figure 1: Complete feasibility map: measured peak VRAM (GB) for all 75 configurations (10 models $\times$ 3 quantizations $\times$ 3 contexts; grey “-” marks the fifteen catalogue-excluded cells: five model-quantization combinations, at three contexts each, notably the 8-bit builds of the two largest models). Colour runs green (light) to red (heavy). Every released configuration fits the 6.99 GB budget.

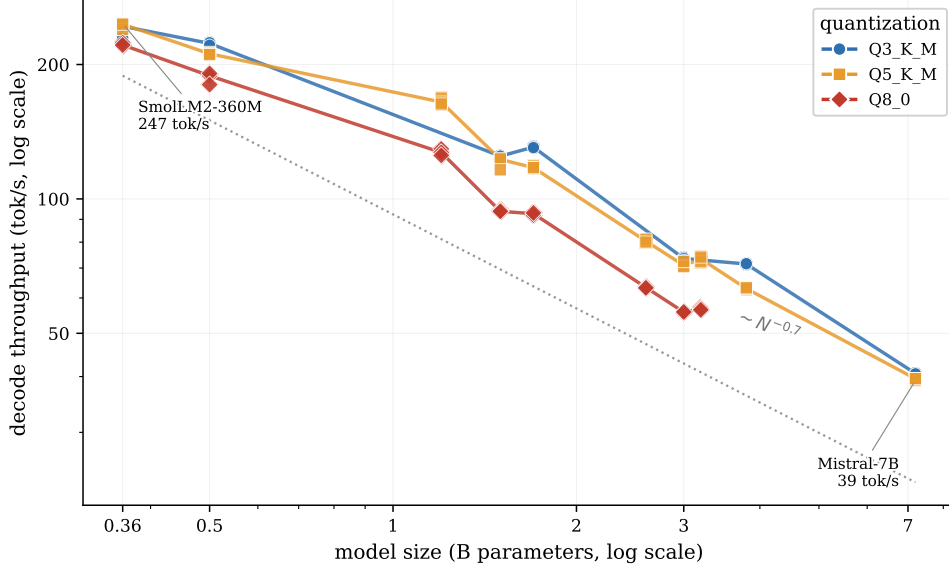


Figure 2: Decode throughput versus model size on log-log axes (all 75 configurations; lines are per-quantization medians). Throughput follows an approximate power law $\propto N^{-0.6}$; heavier quantization is slower at fixed size.

The cost of precision. Moving from 3- to 8-bit weights costs, on average at 4096 context, $1.5\times$ in peak VRAM and $0.80\times$ in throughput (a 20% slowdown), shown per model in Fig. 3. This memory penalty is why the catalogue excludes the 8-bit builds of the two largest models (Mistral-7B, Phi-3.5) a priori; within the released grid every configuration fits, so precision can be chosen for quality rather than forced by memory.

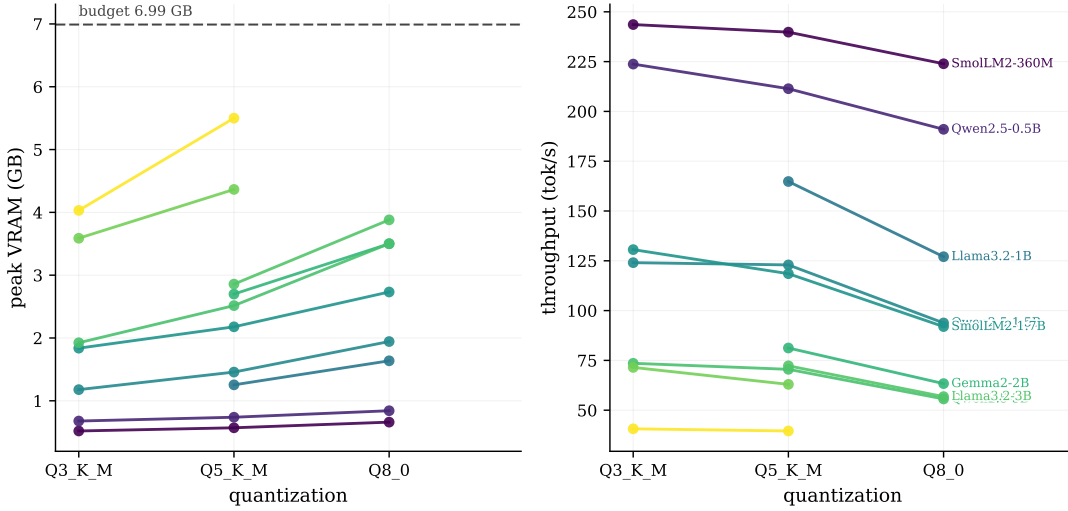


Figure 3: The cost of weight precision at 4096 context (one line per model, colour = size). **Left:** peak VRAM rises $\sim 1.5\times$ from Q3 to Q8; every released configuration stays under the budget. **Right:** throughput falls $\sim 20\%$.

The joint cost of scale. Latency, energy, and memory rise together with model scale, forming a coherent diagonal band in the joint cost space (Fig. 4). Energy and derived latency are near-perfectly correlated in this all-dense pool ($r=0.99$), so a latency budget doubles as a tight energy budget; energy is nonetheless reported as its own measured axis rather than folded into the latency constraint.

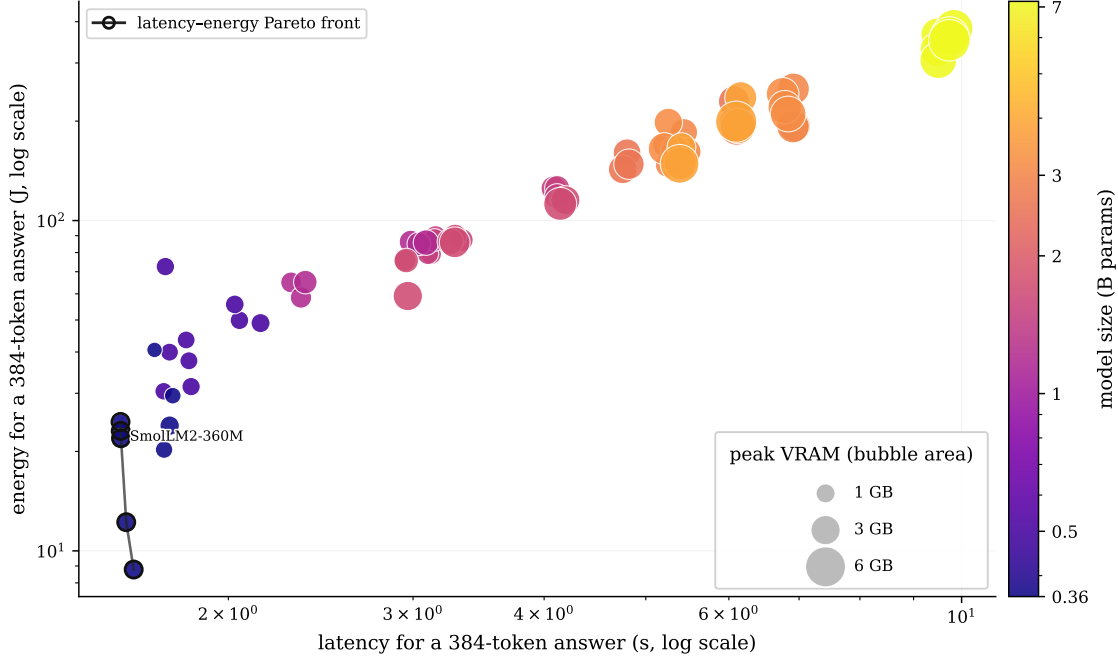


Figure 4: The full cost landscape of all 75 configurations, encoding four dimensions at once: derived latency (x , log), derived energy (y , log), peak VRAM (bubble area), and parameter scale (colour). Cost grows coherently along the diagonal. The black curve is the latency–energy Pareto front (all its points are the smallest model, cheapest on every axis—motivating the need for a capability axis in Section 7).

Cost by model family. Grouping by family confirms the regularity the optimizer relies on: *within* a family, latency and energy scale smoothly and predictably with size (Fig. 5); *across* families at equal size the differences are modest in this all-dense pool (at ≈ 3 B, Gemma-2 answers in 4.8 s against 5.3 s for Llama-3.2-3B and 5.5 s for Qwen2.5-3B). This within-family smoothness is the structure the single-family ablation of §11.1 later exploits.

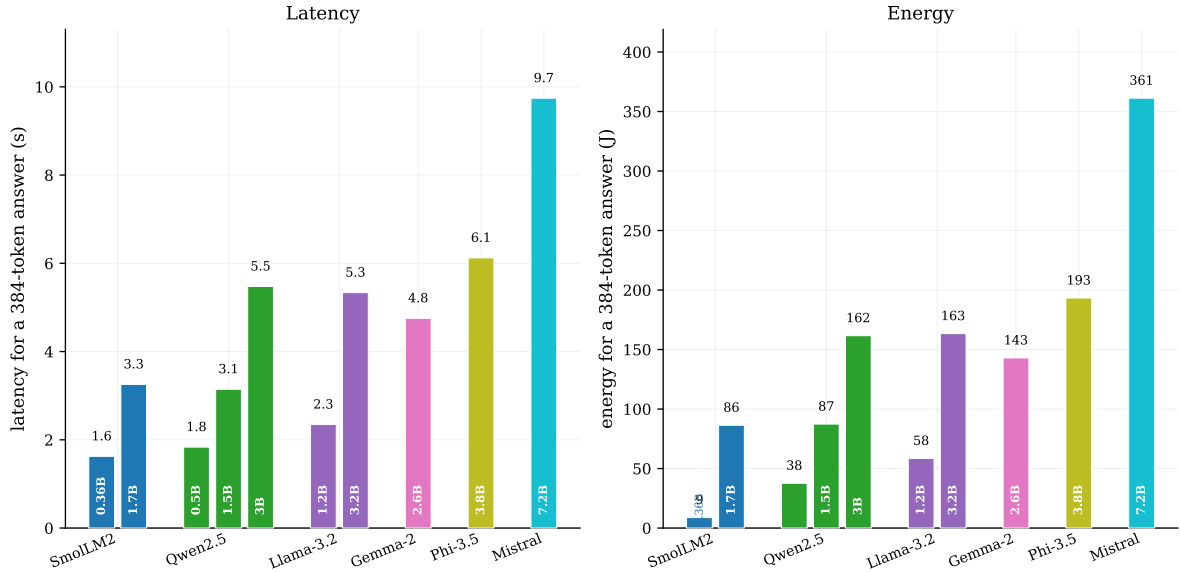


Figure 5: Cost by model family (Q5_K_M, 4096 context); bars grouped by family, the parameter count printed vertically inside each bar. **Left (latency)** and **right (energy)**: both costs grow smoothly with size within every family.

How the metrics relate. Figure 6 summarises the pairwise Pearson correlations across all 75 configurations. Three patterns stand out and recur throughout the paper. Parameter count is strongly tied to peak VRAM (0.85) and inversely to throughput (-0.80), and in this all-dense pool it predicts derived latency almost perfectly (0.96): bigger models are larger in memory and slower, with no architectural exceptions. Energy per token and derived latency are near-perfectly correlated (0.99), and both track parameter count (0.95/0.96)—the empirical basis for treating a latency budget as an approximate energy budget. TTFT is the loosest axis ($|r|$ between 0.51 and 0.75): it is dominated by fixed prefill overhead, though it is not independent of scale.

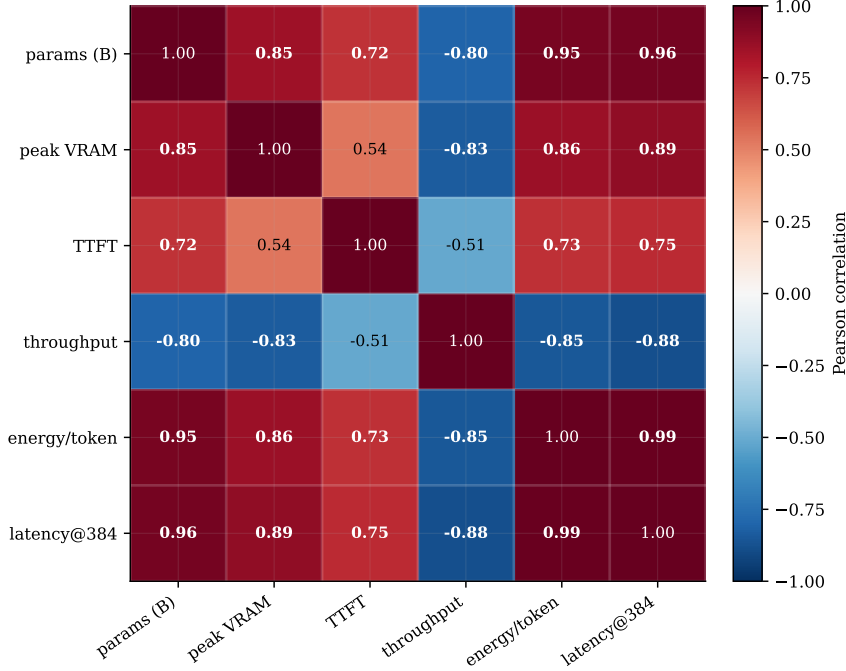


Figure 6: Pearson correlation between the measured performance metrics (and parameter count) across all 75 configurations. Red is positive, blue negative. Note the tight size/VRAM/throughput/latency couplings and the near-perfect energy–latency correlation (0.99); TTFT is the loosest axis.

6 The Allocation Model

6.1 The problem in plain terms

The measurements above tell us what each configuration *costs*. We now use them to make a decision: *which model should each of the eleven agents run?* The agents are not interchangeable. One *dispatcher* reads the query and decides which specialists to invoke; nine *specialists* each answer from their own domain; one *synthesiser* composes the specialists’ outputs into the final reply. Every agent needs a model resident in VRAM, and all of them share a single 8 GB GPU. The allocation must therefore respect two hard limits—a *memory* budget (everything loaded must fit) and a *latency* budget (the end-to-end response must not be too slow)—while making the pipeline as capable as possible.

6.2 The key subtlety: how latency adds up

On a single GPU the pipeline holds *one* loaded instance per model, and the agents on the critical path execute one after another. The system latency is a *sum* along that path, and—crucially—

the specialists are not all on it: the dispatcher activates only a subset per query (that is the point of routing). The dispatcher emits a short routing decision while the specialists and the synthesiser emit full answers. Writing k for the number of activated specialists, and treating the specialists as running on the shared instance one after another (the conservative accounting; see below), the chain is

$$L_{\text{sys}}(k) = \underbrace{L_{\text{disp}}(T_r)}_{\text{router}} + \underbrace{k \cdot L_{\text{spec}}(T_g)}_{k \text{ activated specialists}} + \underbrace{L_{\text{synth}}(T_g)}_{\text{synthesiser}}, \quad (7)$$

where $T_r \approx 15$ routing tokens and $T_g = 384$ answer tokens, and $L(\cdot)$ uses the measured per-token rate of the chosen model (Eq. 6). Because the specialists do the same kind of work they share one model, so the k activated terms collapse to $k \cdot L_{\text{spec}}$. This is what makes the allocation non-trivial: every distinct model on the critical path *adds* latency, while capacity wants *distinct* models—a genuine tension between the two budgets.

Why a sum, and the batched alternative. A specialist stage cost of $k \cdot L_{\text{spec}}$ describes the activated specialists executing serially on the shared instance—the *conservative worst case* on latency, since several specialists sharing one loaded model cannot run truly in parallel on one GPU. We adopt this sum (7) as the primary model: it is consistent with the load-once memory accounting, and any deployment that instead *batches* the shared specialists into a single forward pass can only *reduce* the specialist term. We test that opposite extreme—an exact *batched* (concurrent-stage) model in which the stage cost is a single specialist generation $\max_{s \in \mathcal{A}} L_{\text{spec}}$, the `worst_case` latency mode of the released solver—in §10.5, and find the optimal allocation unchanged across the whole range from serial to batched over the realistic budget. Since which specialists fire is query-dependent, we do not fix k ; we trace the frontier parametrically over $k \in \{1, 3, 5, 9\}$ (the calibration set activates 2.68 specialists on average, median 2, maximum 6).

6.3 What we maximize, and the honest caveat

The pipeline has three generation roles: the dispatcher d , one specialist model s shared by all specialists, and the synthesiser y . We maximize the pipeline’s *distinct-model capacity*: the summed parameter count of the *distinct* models placed on these three slots, each model counted *once* even if two slots share it. This is the corrected objective: an earlier version summed parameters per agent, which counted a shared model many times and rewarded placing one large model everywhere; counting distinct models removes that degeneracy and forces the optimizer to trade memory for variety.

This remains a deliberate proxy, not measured answer quality. The model answers “how much distinct model can the pipeline afford at a given latency,” not “how good are its answers.” We use parameter count only as a stand-in for capability; §5 showed cost scales coherently with size in this dense pool, but cost is not capability—§8 will show the proxy is actively misleading for specialist quality. Substituting measured per-role quality for parameter count—once the quality campaign completes—turns the same program into a quality-aware allocator.

6.4 The mixed-integer linear program

Sets. Three role-slots $\mathcal{R} = \{d, s, y\}$ (dispatcher, shared specialist, synthesiser). Candidate configurations $\mathcal{C}$, each a triple $c = (\text{model}, \text{quant}, \text{context})$; let $\mathbf{m}(c)$ be its model and $\mathcal{M}$ the set of models. Configurations sharing weights (same model and quant, differing in context) form a group $g \in \mathcal{G}$.

Measured data. Each c has parameter count $\pi_{\mathbf{m}(c)}$ (B), peak VRAM μ_c (GB), and two derived

latencies— ℓ_c^r at T_r tokens and ℓ_c^g at T_g tokens (Eq. 6). Budgets: VRAM $M = 6.99$ GB, latency ε , and the activation count k .

Decision variables.

$$x_{r,c} \in \{0, 1\} : \quad \text{role } r \text{ runs configuration } c; \quad (8)$$

$$z_c \in \{0, 1\} : \quad \text{configuration } c \text{ is loaded in VRAM}; \quad (9)$$

$$u_m \in \{0, 1\} : \quad \text{model } m \text{ is used by some role (for capacity)}. \quad (10)$$

Objective—maximize distinct-model capacity:

$$\max \sum_{m \in \mathcal{M}} \pi_m u_m. \quad (11)$$

Constraints. In words: (12) each role runs one configuration; (13) a role may run a configuration only if it is loaded; (14) at most one context per weight-sharing group is loaded; (15) loaded configurations fit in VRAM, each counted once; (16) a model counts toward capacity only if some role uses it; (17) the serial latency chain (7) stays within budget.

$$\sum_{c \in \mathcal{C}} x_{r,c} = 1 \quad \forall r \in \mathcal{R} \quad (12)$$

$$x_{r,c} \leq z_c \quad \forall r \in \mathcal{R}, c \in \mathcal{C} \quad (13)$$

$$\sum_{c \in g} z_c \leq 1 \quad \forall g \in \mathcal{G} \quad (14)$$

$$\sum_{c \in \mathcal{C}} \mu_c z_c \leq M \quad (15)$$

$$u_m \leq \sum_{r \in \mathcal{R}} \sum_{c: m(c)=m} x_{r,c} \quad \forall m \in \mathcal{M} \quad (16)$$

$$\sum_c \ell_c^r x_{d,c} + k \sum_c \ell_c^g x_{s,c} + \sum_c \ell_c^g x_{y,c} \leq \varepsilon \quad (17)$$

Properties. Constraint (17) is the serial chain of Eq. (7): the specialist term $k \sum_c \ell_c^g x_{s,c}$ charges the k activated specialists running one after another on the shared instance (the conservative worst case; §10.5 shows the optimum is unchanged under the batched alternative). The program is linear and tiny—three role-slots over ~ 30 model-quant groups—and solves in well under a second with CBC. It is genuinely constrained on *both* axes: capacity (11) pushes toward three distinct models, while memory (15) and the serial latency (17) push back. A *quality-aware* version replaces π_m in (11) with measured per-role quality and gates the specialist term by the dispatcher’s routing accuracy; that gating is bilinear (a product of two binaries) and needs McCormick linearization (§10).

7 Optimization Results

This section answers Q1’s second half: given the measured costs, what does the proxy-optimal allocation look like, and is the optimization even necessary? We solve the MILP over a sweep of the latency budget ε for each activation level $k \in \{1, 3, 5, 9\}$, from the smallest feasible ε (the fastest configuration on all three slots) upward. The result is the **system capacity–latency frontier** (Fig. 7).

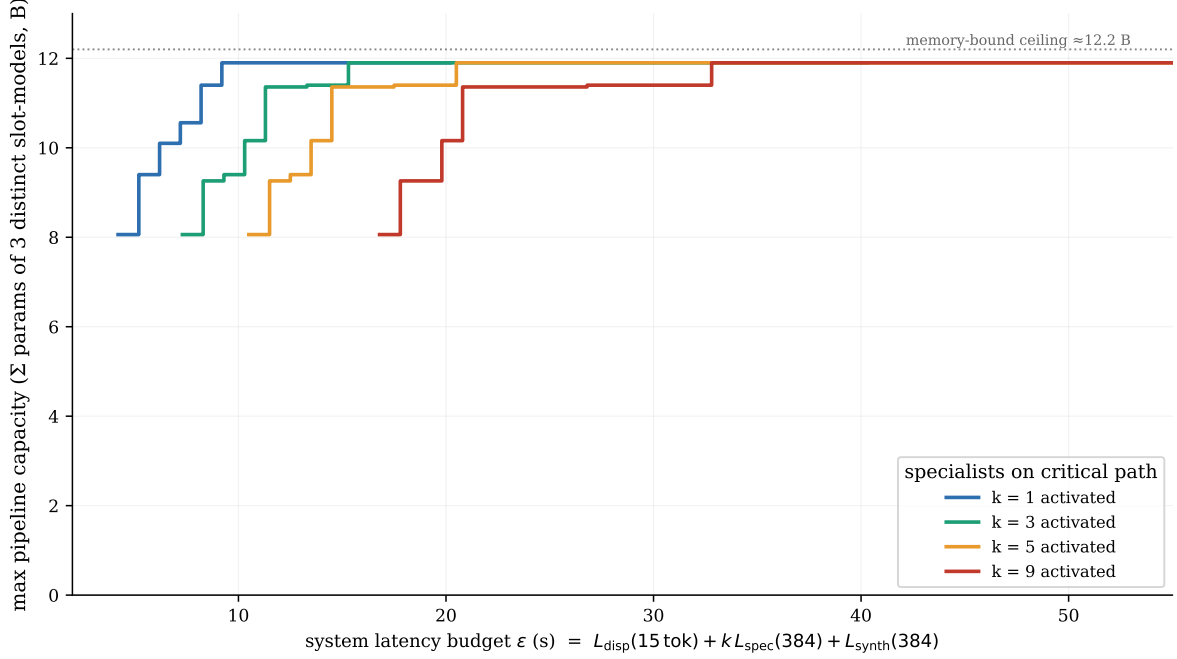


Figure 7: System capacity–latency frontier under the 6.99 GB budget, for $k \in \{1, 3, 5, 9\}$ activated specialists (serial chain). Capacity (sum of the three distinct slot-models’ parameters) rises in a staircase to a memory-bound ceiling of ≈ 11.9 B. Increasing k shifts the whole frontier rightward: the same capacity costs more latency because each activated specialist adds to the serial chain.

The ceiling is set by memory, not latency. The frontier plateaus at ≈ 11.9 B for every k once ε is large enough. This ceiling is the largest sum of three distinct models that fits in 6.99 GB—the optimizer loads three models totalling 11.9 B and uses 6.8 GB at the plateau. Memory, not the latency chain, is the binding constraint at capacity; the latency budget only governs *how soon* along ε the ceiling is reached. (This corrects an earlier formulation in which double-counted capacity and an inconsistent latency model produced an illusory 72 B figure; under coherent accounting the realisable capacity is roughly six times smaller.)

The cost of an active pipeline (the role of k). The four curves quantify the serial-chain penalty. With a single activated specialist the full 11.9 B ceiling is reached already at $\varepsilon \approx 9$ s; it requires $\varepsilon \approx 15$ s at $k=3$, ≈ 21 s at $k=5$, and ≈ 33 s at $k=9$. Because the calibration set activates 2.68 specialists on average, the $k=3$ curve is the operationally relevant one: a full-capacity pipeline needs a latency budget of roughly 15 s under realistic routing, not the ~ 9 s a single-specialist reading would suggest. (§10.5 shows that batching the shared specialists collapses this penalty—the same allocation then reaches full capacity at a much tighter budget—without changing the optimal allocation itself.)

What the optimizer does (and what is *not* an emergent rule). Two behaviours are worth noting, but neither is a deep discovery. First, the dispatcher is consistently assigned a *large* model, because with only $T_r \approx 15$ routing tokens its latency is ~ 0.4 s regardless of size, so it is nearly free to make the dispatcher large and gain its parameters toward capacity. This is a direct consequence of the realistic per-role token counts, not an independent design principle. Second, the optimizer always selects exactly three distinct models (one per slot); there is no incentive to load unused models. Both behaviours follow mechanically from the objective and the batched latency model.

Frontier detail and the latency chain. Figure 8 traces the $k=3$ frontier with the specialist-tier transitions annotated, against the faint $k \in \{1, 5, 9\}$ curves; Table 3 lists its knots. The allocation climbs by enlarging the specialist tier (SmolLM2-360M $\rightarrow$ Qwen-0.5B $\rightarrow$ Llama-1B $\rightarrow$ SmolLM2-

1.7B) while the dispatcher stays on a large model (cheap at 15 routing tokens) and the synthesiser acts as the swing slot, absorbing spare budget. Figure 9 decomposes the critical-path latency at each knot: the dispatcher term is negligible (~ 0.4 s), the $k=3$ specialist term dominates and grows as larger specialists are afforded, and the synthesiser fills the remaining budget; the stacked totals track ε , confirming the latency constraint binds at the knots while memory binds at the ceiling.

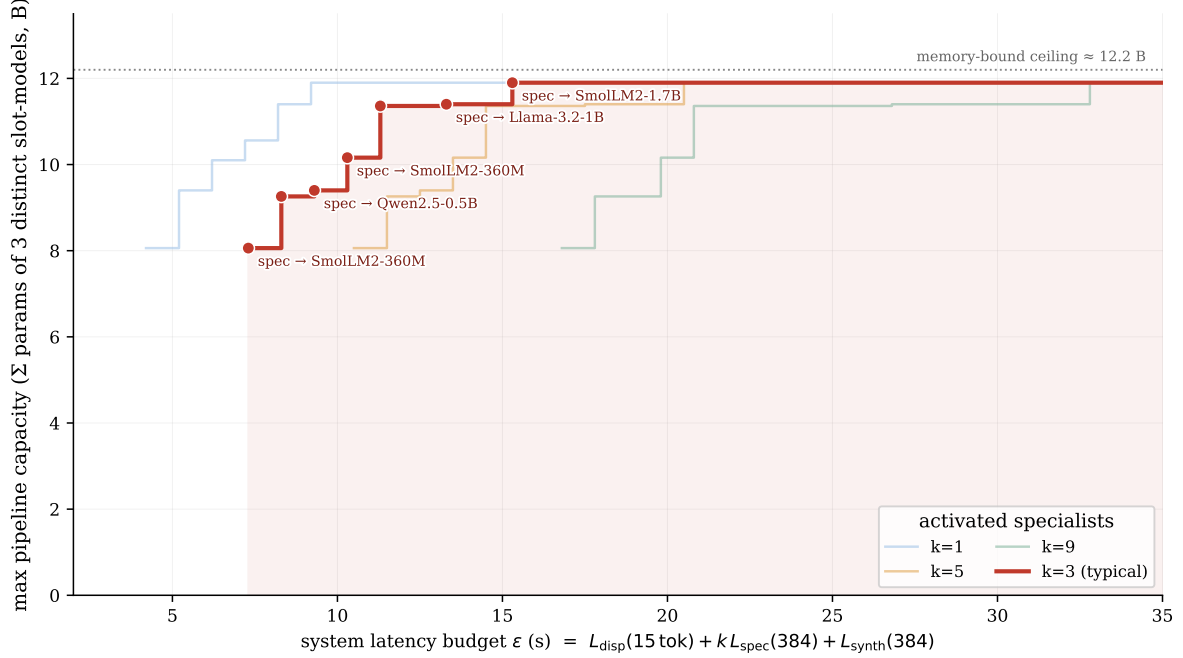


Figure 8: System capacity–latency frontier (serial chain) with the $k=3$ curve highlighted and annotated at each specialist-tier change; faint curves are $k \in \{1, 5, 9\}$. Capacity is the sum of the three distinct slot-models’ parameters; it rises in a staircase to a memory-bound ceiling of ≈ 11.9 B. Larger k shifts the curve rightward (more activated specialists cost more serial latency for the same capacity).

Table 3: Knots of the $k=3$ capacity–latency frontier (full data in `syscap3.frontier.csv`). “Specialist” is the model shared by all specialists. Capacity counts the three distinct slot-models once each; VRAM is the load-once total. Note the memory-bound ceiling at 11.9B and the large dispatcher (cheap at 15 routing tokens). Listed configurations are the *canonical* optima: among objective-equal solutions the released code selects the minimum-latency, then minimum-memory configuration, so the table is machine-independent. ε values are under the serial $k=3$ chain; §10.5 shows the same allocations hold under batching, at tighter budgets.

ε (s)	capacity (B)	VRAM (GB)	dispatcher	specialist	synthesiser
7.3	8.1	5.16	Mistral-7B	SmolLM2-360M / Q5	Qwen2.5-0.5B
8.3	9.3	5.82	Mistral-7B	SmolLM2-360M / Q5	SmolLM2-1.7B
9.3	9.4	6.04	Mistral-7B	Qwen2.5-0.5B / Q3	SmolLM2-1.7B
10.3	10.2	6.81	Mistral-7B	SmolLM2-360M / Q5	Gemma-2-2B
11.3	11.4	6.82	Mistral-7B	SmolLM2-360M / Q5	Phi-3.5-mini
13.3	11.4	6.67	Mistral-7B	Llama-3.2-1B / Q5	Qwen2.5-3B
15.3	11.9	6.84	Mistral-7B	SmolLM2-1.7B / Q3	Qwen2.5-3B

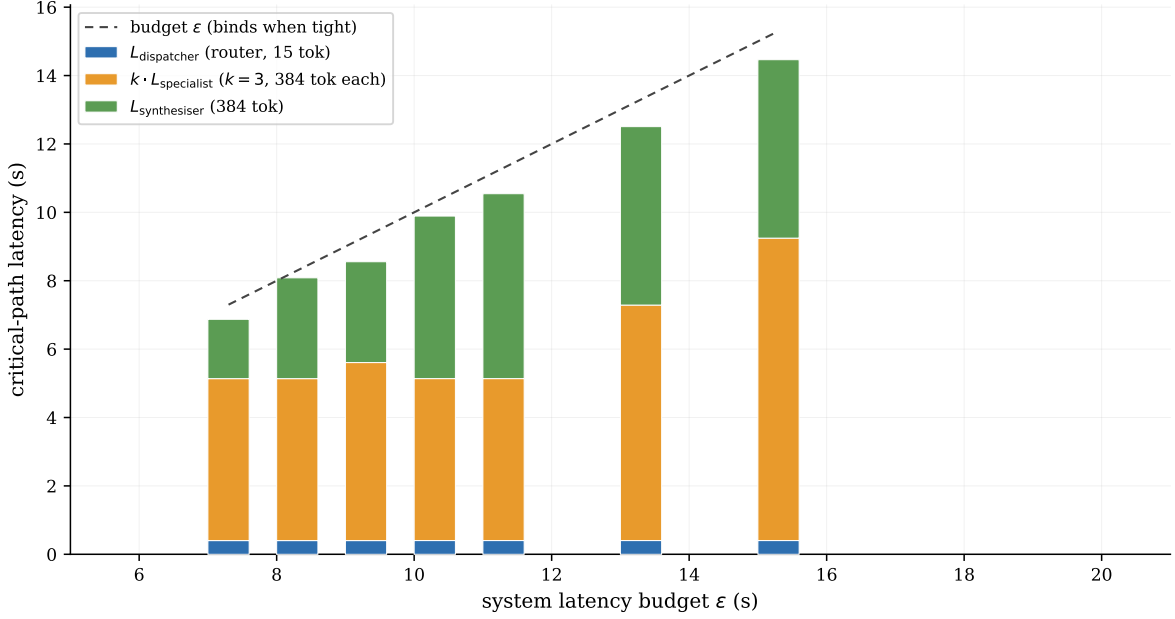


Figure 9: How the latency budget is spent along the serial critical path at each $k=3$ knot: dispatcher (router, 15 tokens) + $k L_{\text{spec}}$ (384 tokens each) + synthesiser (384 tokens). The dispatcher term is negligible; the specialist term dominates and grows with the specialist tier; the synthesiser absorbs the slack. The stacked totals track ε , confirming the latency constraint binds at the knots.

7.1 Does the optimization beat a trivial heuristic? (performance-only)

A strict test of whether the MILP earns its keep: we compare it against a *greedy* heuristic (independently give each slot the largest-parameter model whose configuration keeps the chain within ε and fits memory) and a *uniform* baseline (the single largest model placed on all three slots). Across 91 (ε, k) points (Fig. 10), the honest result is sobering: **the MILP ties the greedy heuristic at 84/91 points (92%) and strictly beats it at only 7**, all in a narrow mid-latency band, by 0.1–1.2 B (mean 0.56 B). The uniform baseline is far worse everywhere. The reason is structural: with the parameter-count objective, the best choice per slot is simply “the largest model that fits the slot’s latency share,” so the problem is *sortable* and a one-line greedy reproduces it except where the memory and latency constraints couple tightly. **On performance-only data, the optimization adds little over a sort, and we say so.**

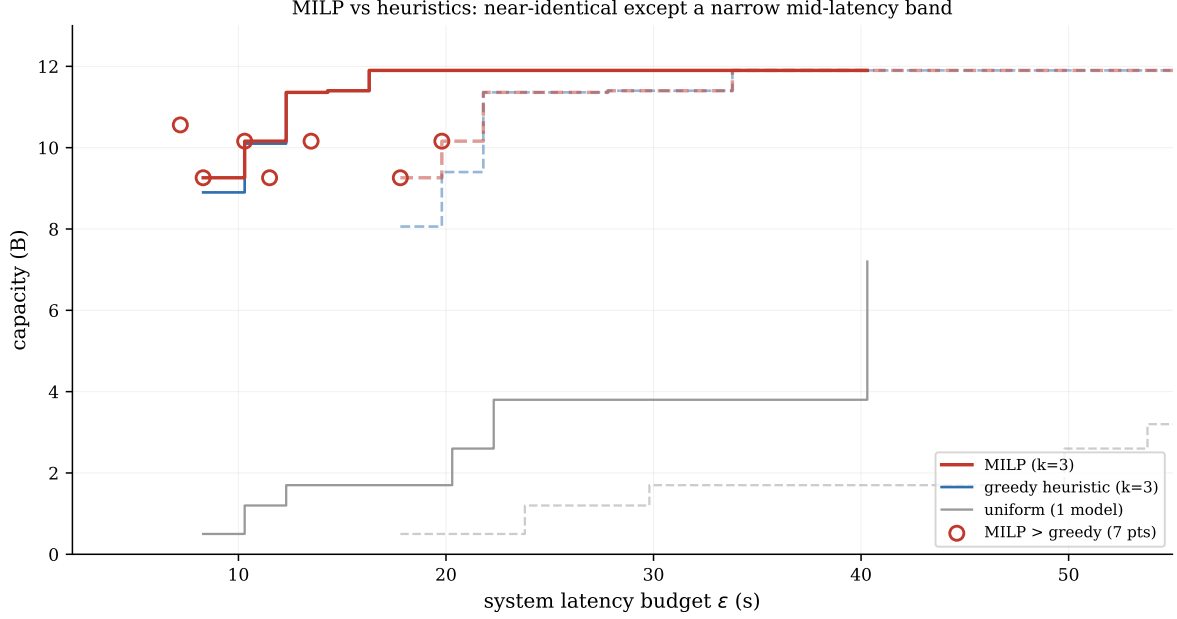


Figure 10: Performance-only objective (parameter capacity): MILP versus baselines (solid $k=3$, dashed $k=9$). The MILP (red) and greedy heuristic (blue) are nearly indistinguishable; circles mark the seven points where the MILP strictly wins. The uniform single-model baseline (grey) is far below. When the objective is sortable, optimization adds little over a greedy sort.

7.2 When optimization matters: heterogeneous specialists

The greedy heuristic ties the MILP only because the parameter objective is sortable. The realistic objective is not. The nine specialists cover *different* legal domains (testamentary vs intestate succession, consensual vs contested separation, minor protection, family mediation, ...), and a model that is strong on one domain may be weak on another—“pick the largest” is no longer “pick the best.” To quantify the effect we attach to each (model, domain) pair an *affinity* $A_{m,\delta} \in [0, 1]$ and maximize the summed affinity of the k activated specialist-domains plus the synthesiser’s quality (and a small routing-quality term for the dispatcher), under the same memory and serial-latency constraints. The specialist slots are now *distinct* (slot s_j serves domain δ_j), so the MILP must co-optimize which model serves which domain against the shared budgets—a genuinely combinatorial problem.

Honesty: these affinities are synthetic. We do not yet have measured per-domain quality (it requires the quality campaign, still running), so $A_{m,\delta}$ here is *synthetic but plausible*: each model has a latent skill that increases with log-parameters but saturates and carries model-specific noise, perturbed by ± 0.12 domain-specific deviations so that a smaller model can top a given domain. This subsection is therefore a *demonstration of when optimization is needed*, on synthetic affinities, not a measured result; when the quality table lands, $A_{m,\delta}$ is replaced by measured per-domain quality and the same MILP runs unchanged.

Under this heterogeneous objective the picture inverts (Fig. 11): **the MILP beats the greedy heuristic on 84/91 points (92%)**, and—tellingly—the advantage grows with the number of activated specialists, from 25% at $k=1$ (a near-sortable single-domain problem, where greedy is adequate) to 94% at $k=3$ and 100% at $k \geq 5$. The mechanism is exactly what the greedy cannot see: with several domains competing for one shared memory budget and one serial latency budget, the best global assignment trades a slightly worse model on one domain for a much better one on another—a joint decision a per-slot sort cannot make. This is the regime in which the MILP earns its keep, and it is the regime the production system actually inhabits once

answer quality, not parameter count, is the objective.

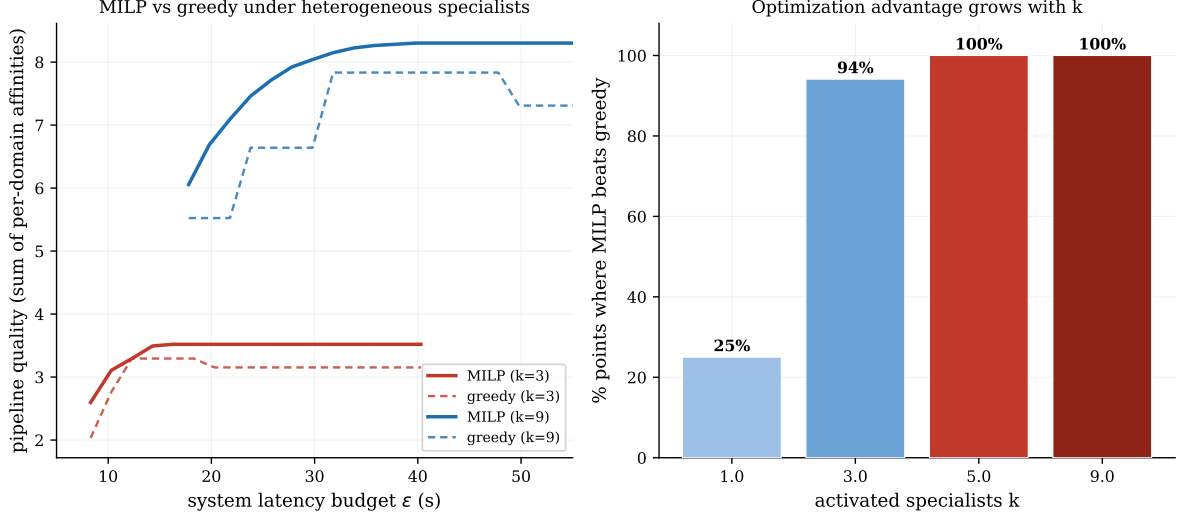


Figure 11: Heterogeneous-specialist objective (*synthetic* per-domain affinities, clearly labelled). **Left:** pipeline quality versus latency budget for $k=3$ and $k=9$; the MILP (solid) dominates the greedy heuristic (dashed) at every budget. **Right:** the fraction of points where the MILP strictly beats greedy rises with the number of activated specialists k , from 25% ($k=1$, near-sortable) to 100% ($k \geq 5$). When the objective is not sortable, optimization is necessary, not marginal.

7.3 Sensitivity of the frontier to measurement noise

Lacking repeated raw runs, we test robustness by Monte-Carlo sensitivity: we perturb every configuration’s throughput and TTFT by independent lognormal noise ($\sigma=10\%$, a realistic laptop-GPU thermal-drift figure), recompute derived latency, and re-solve at $k=3$ over 25 draws (Fig. 12). The frontier is stable: the 10–90 percentile band is ± 0.5 B around the median and collapses to zero at high ϵ , where memory binds hard and the capacity ceiling is noise-independent. Tier transitions shift by at most a fraction of one ϵ step. We label this a sensitivity analysis, not a confidence interval from real replicates.

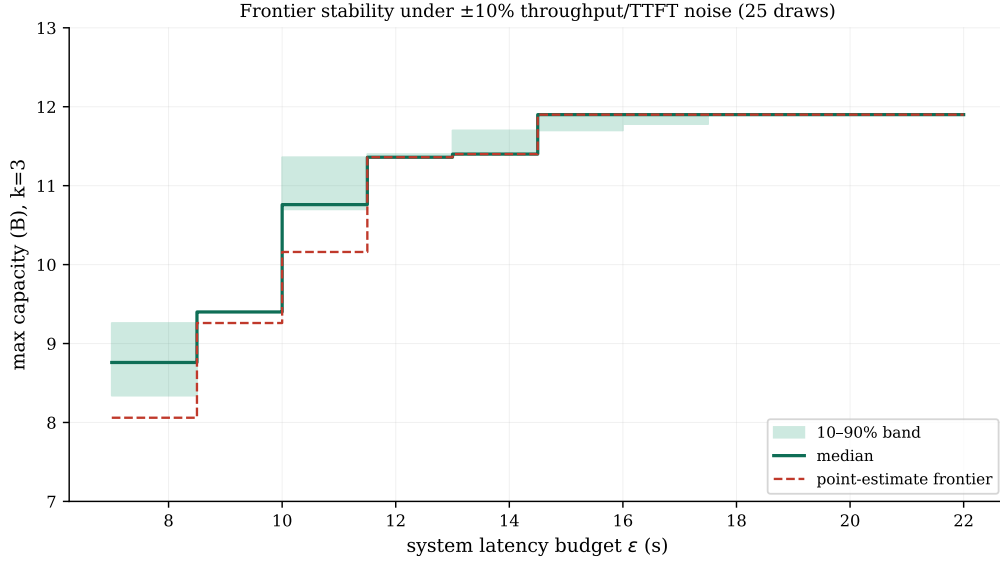


Figure 12: Frontier stability at $k=3$ under $\pm 10\%$ throughput/TTFT noise (25 draws). The 10–90% band (shaded) is narrow and vanishes at high ϵ ; the point-estimate frontier (dashed) sits inside it. Capacity tiers are robust to plausible measurement variation.

8 Measured Quality Results (All Roles)

The frontier above optimizes a parameter-count proxy. We now report the first results of the *measured*-quality campaign, which scores each agent’s output with judge-free, embedding-based metrics (routing F1 for the dispatcher; RAG faithfulness, answer relevancy, context precision/recall, and correctness for the specialists). These results let us answer the question the proxy cannot: *is the highest-capacity allocation actually the highest-quality one?*

Status of the campaign. The campaign covers all eleven agents over 75 of 75 configurations and 129 query instances, with 7959 scored agent–query–config records: 3900 for the dispatcher, 3439 across the nine specialists, and 620 for the synthesiser. All three roles are now analyzed below. Coverage is not yet exhaustive—a few weight-sharing groups and some lower-frequency domains remain under-sampled—so per-domain confidence intervals will tighten as the campaign completes, but the qualitative findings are stable.

8.1 Quality does not track parameter count

Across specialist configurations, measured quality correlates only weakly with parameter count (Pearson $r=0.54$ at the configuration level, 0.31 at the per-query level) and, more importantly, the relationship is *non-monotone* (Fig. 13). The single best specialist model is **Llama-3.2-1B** (mean quality 0.74), which *outscores every larger model*, including Mistral-7B (0.72) at $6\times$ the parameters, and Phi-3.5. Conversely, Qwen-0.5B is the *worst* model despite not being the smallest. Parameter count is thus an actively misleading capability proxy for the specialist role.

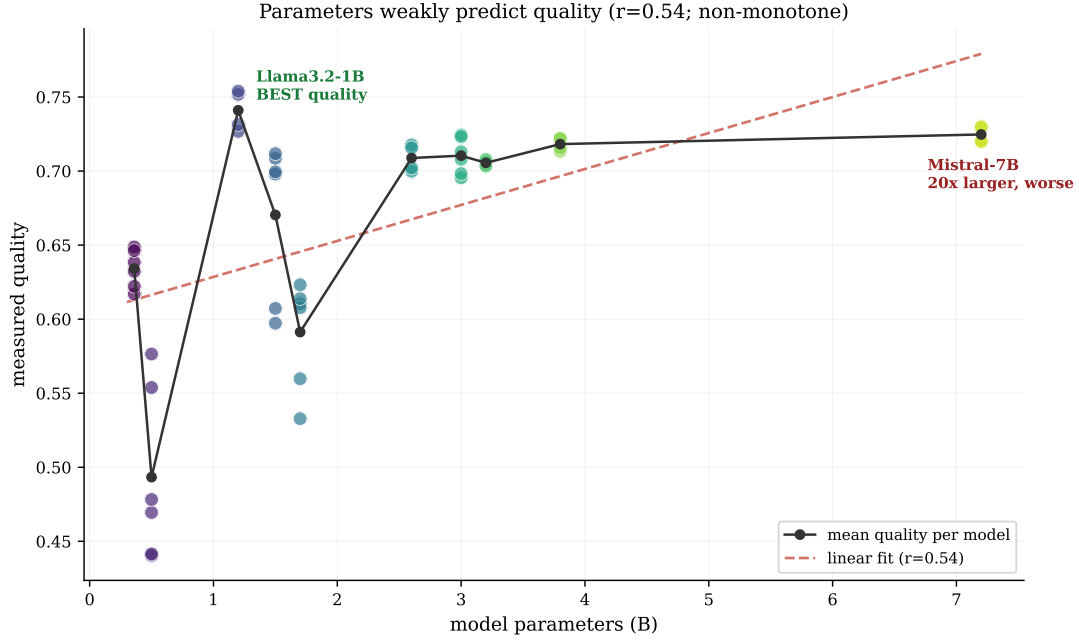


Figure 13: Measured specialist quality versus parameter count. The per-model mean (black) is non-monotone; the linear fit is weak ($r=0.54$). Llama-3.2-1B is the best model overall, beating Mistral-7B at a fraction of the size.

8.2 The quality–latency frontier is dominated by small models

Building the Pareto frontier of measured quality versus latency (Fig. 14), **only 4 of 50 specialist configurations are non-dominated, and none exceeds 1.2 B parameters**. Every large model (Mistral-7B, Phi-3.5) is strictly dominated: it is matched or beaten in quality by Llama-3.2-1B while costing 4–6 $\times$ the latency and up to 6 $\times$ the VRAM. The quality frontier is therefore the near-opposite of the parameter-capacity frontier of §7.

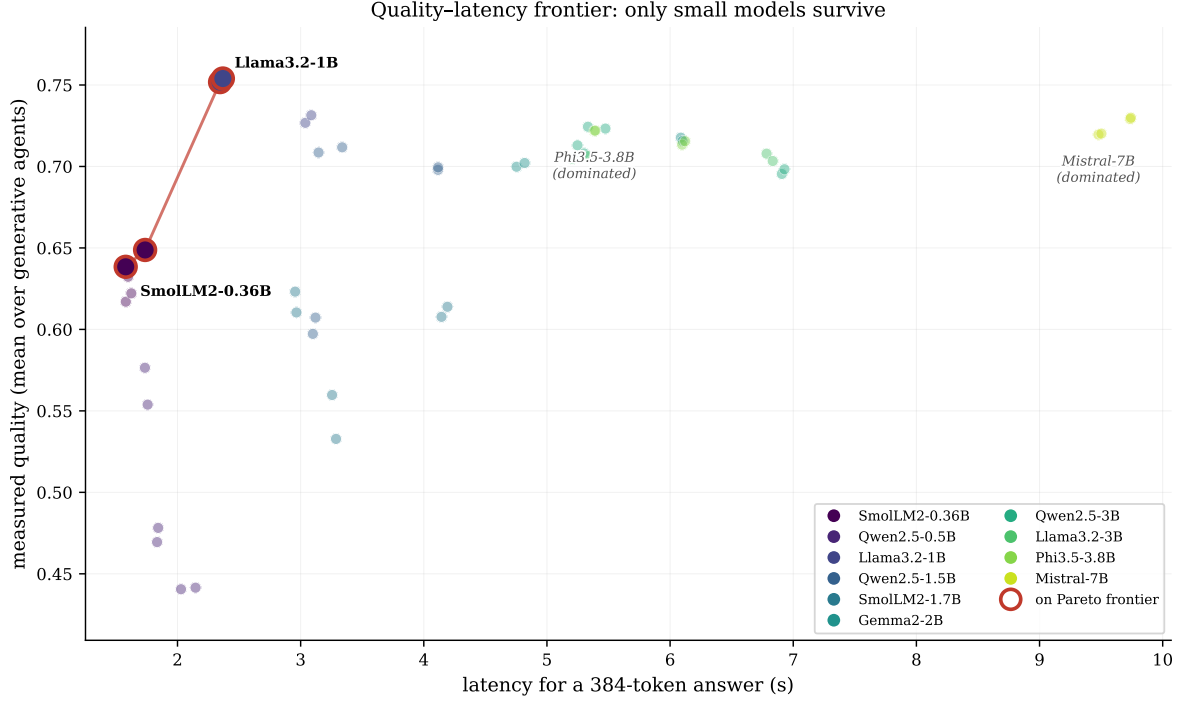


Figure 14: Measured quality versus latency for all specialist configurations. Only four small-model configurations lie on the Pareto frontier (red rings); the large models sit in the dominated upper-right region (same quality, far higher latency).

8.3 Validation: the highest-capacity allocation is sub-optimal in quality

We can now validate the parameter-proxy allocator directly. For each latency budget we compare the model the capacity-maximizing policy (§7) would place in the specialist slot against the measured-quality-optimal choice (Fig. 15). The capacity policy is **sub-optimal at every one of the 105 budget steps tested**, by a mean of 0.051 and up to 0.131 quality points. As the budget grows it spends the extra latency on ever-larger models (SmolLM2-1.7B → Gemma-2 → Phi-3.5 → Mistral-7B) without ever matching the quality that Llama-3.2-1B delivers cheaply. This is direct, measured evidence that maximizing capacity (or parameters) is the wrong objective, and it quantifies the quality left on the table by a proxy-based allocator—the gap a quality-aware MILP would close.

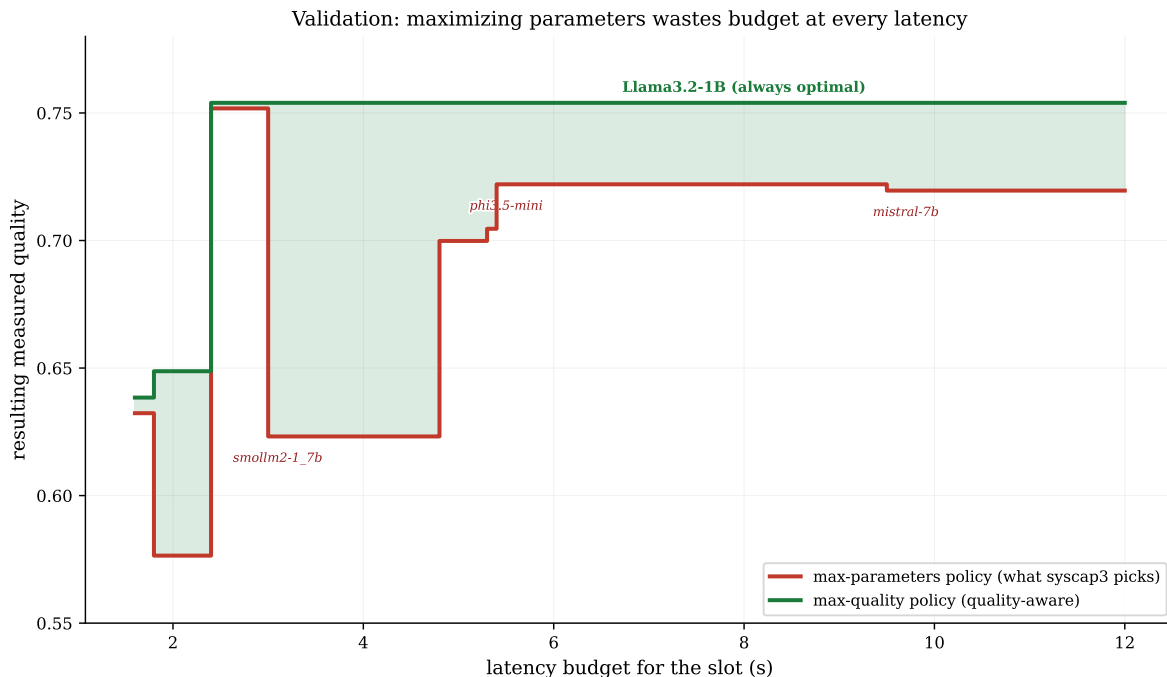


Figure 15: Validation of the capacity proxy. At each latency budget, the quality of the model chosen by the capacity-maximizing policy (red) versus the measured-quality-optimal model (green). The capacity policy is dominated at every budget; the shaded region is the quality forgone.

8.4 Held-out validation: the small-specialist finding is not in-sample overfitting

The quality-aware allocator selects the specialist using measured quality, and a fair objection is that the resulting choice could be over-fit to the small evaluation set—the reported quality being optimistic because the same queries both select and score the model. We test this with a *leave-one-domain-out* (LODO) protocol, which is stricter than a random query split because it withholds an entire legal domain: for each of the nine domains in turn, we select the best specialist model using only the queries of the *other eight* domains, then evaluate that model on the held-out domain it was not allowed to see.

The finding survives the split cleanly. The model selected on the training domains is Llama-3.2-1B in **9 of 9 folds**; the choice never depends on the held-out domain. Its held-out quality tracks the per-fold oracle (the best model chosen *with* access to the held-out domain) almost exactly—a mean generalization gap of 0.001 and at most 0.008 quality points across all folds. And the small-model advantage itself generalizes: on the held-out domain the train-selected Llama-3.2-1B beats Mistral-7B in 7 of 9 folds (mean +0.018), losing narrowly only on the two domains where a larger model is genuinely best (family mediation and consensual separation, both Mistral-7B), consistent with the per-domain heterogeneity reported next. The quality-aware optimum is therefore a property of the measured data, not an artifact of selecting and scoring on the same queries; the residual caveat is the absolute sample size (§13), not the selection procedure.

8.5 Specialist evaluation by domain

The nine specialists cover distinct legal domains, and the best model differs by domain (Fig. 16)—the structural premise of the heterogeneous formulation (§7), now measured rather than synthetic. Llama-3.2-1B is the best specialist in 7 of 9 domains, but not all: family mediation (*ne-*

goiazione) and consensual separation are both best served by Mistral-7B. The quality spread across models within a domain is large (e.g. 0.47–0.76 for *volontaria giurisdizione*), confirming that model choice per domain matters and is not reducible to a single ranking.

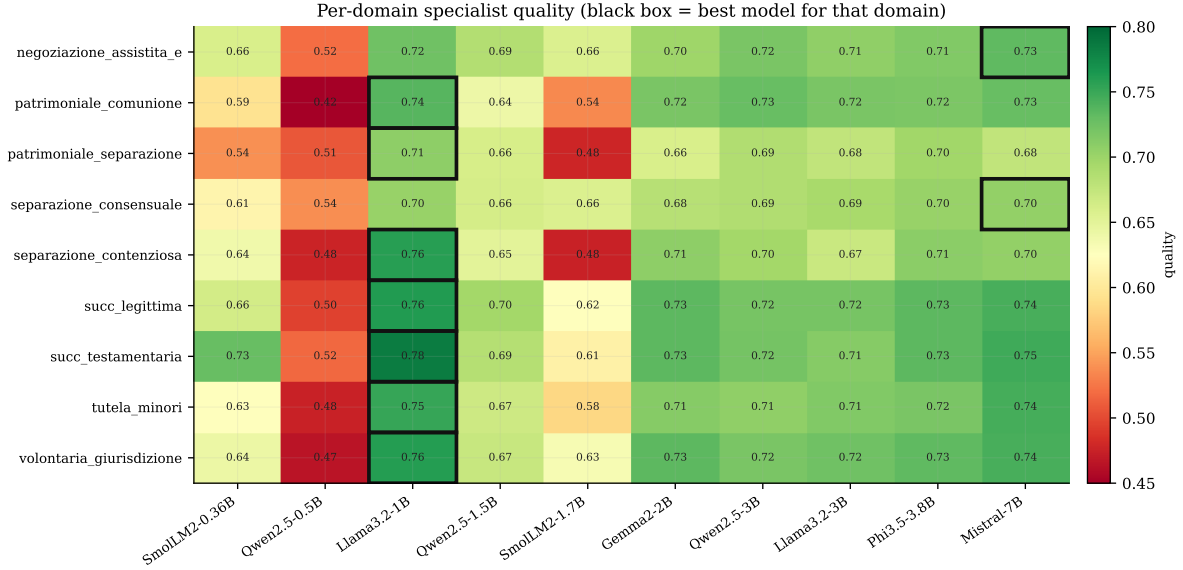


Figure 16: Per-domain specialist quality for every model (black box = best model in that domain). Llama-3.2-1B (the strong column) wins most domains; Mistral-7B wins the other two, illustrating genuine per-domain heterogeneity.

Statistical caveat (small per-domain samples). These per-domain rankings rest on few queries each: only 2 of the 9 domains (*succ. legittima*, $n=22$; *succ. testamentaria*, $n=14$) have $n \geq 10$; the other 7 have $n \in \{3, \dots, 9\}$. We therefore report 95% bootstrap confidence intervals on each domain’s best-model quality (Fig. 17) and flag the under-powered domains explicitly. The intervals are wide where n is small (e.g. *separazione contenziosa*, $n=4$, CI [0.68, 0.83]), so individual per-domain “best model” claims with $n < 10$ should be read as suggestive, not conclusive. The *pooled* signal is nonetheless strong: pooled over Llama-3.2-1B’s 282 specialist records (of 3439 in total), its lead is tight (CI [0.743, 0.764]), and it is the best or statistically-tied model in 7 of 9 domains. The robust claim is the pooled and cross-role one (the three regimes); the per-domain winners are indicative.

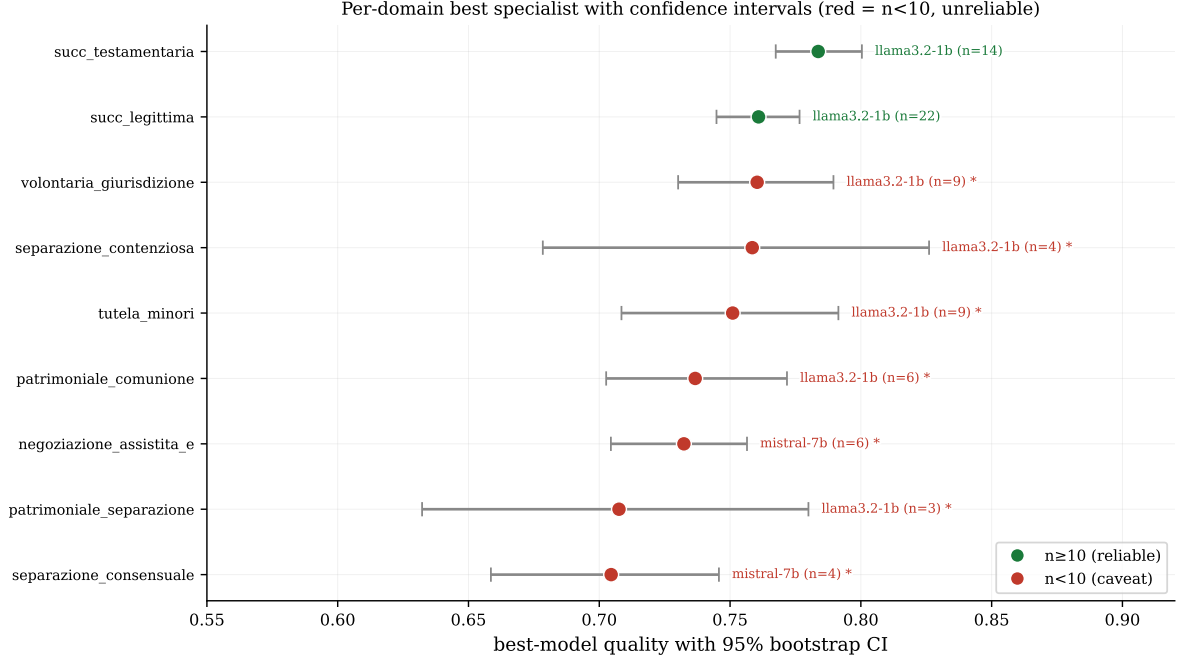


Figure 17: Per-domain best-specialist quality with 95% bootstrap confidence intervals. Green = $n \geq 10$ (reliable); red = $n < 10$ (caveat). Intervals widen sharply at small n ; the pooled specialist signal (not shown here) is far tighter.

8.6 Dispatcher evaluation: routing F1 rewards larger models

The dispatcher is scored by routing F1 (predicted vs. expected specialist set). Routing quality increases with model size (Fig. 18), led by Phi-3.5 (0.65), Mistral-7B (0.59), and Gemma-2 (0.57), while the sub-2 B models trail at 0.18–0.39. One model is excluded from the trend reading: across its 312 dispatcher records, Llama-3.2-3B emits a *constant* specialist set independent of the query, a routing-format failure that yields $F_1=0.06$ —a degenerate outlier, not a noisy estimate. Routing—a short, structured classification over the query—benefits from larger models, the opposite of the generative specialist role.

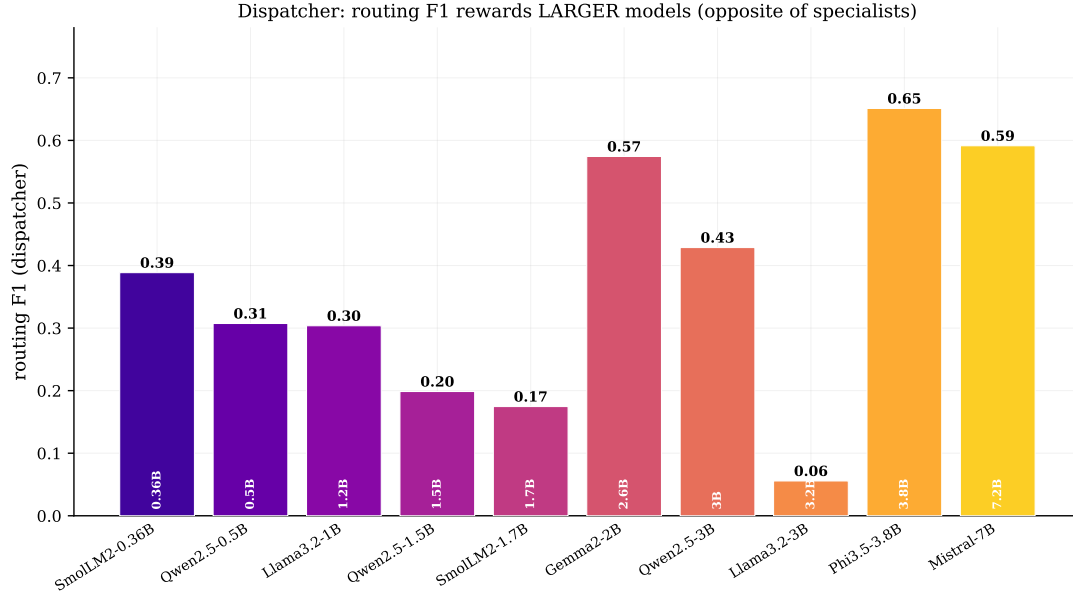


Figure 18: Dispatcher routing F1 by model. Unlike the specialists, routing quality rises with size (Phi-3.5 and Mistral-7B lead). (Llama-3.2-3B is a degenerate outlier: it routes every query to the same fixed specialist set; see text.)

8.7 Synthesiser evaluation: quality rises with size

The synthesiser pass is now sufficiently advanced to analyze (620 scored records over 25 configurations and all 25 calibration queries). Its behaviour is the *opposite* of the specialists’ and matches the dispatcher’s: synthesis quality rises with model size, nearly monotonically (Fig. 19), and the best synthesiser is the largest model, **Mistral-7B** (quality 0.77), ahead of Phi-3.5 (0.73) and Gemma-2 (0.72), with the sub-2 B models trailing at 0.58–0.65. Composing a coherent final answer from several specialist outputs evidently benefits from a larger model, much as routing does.

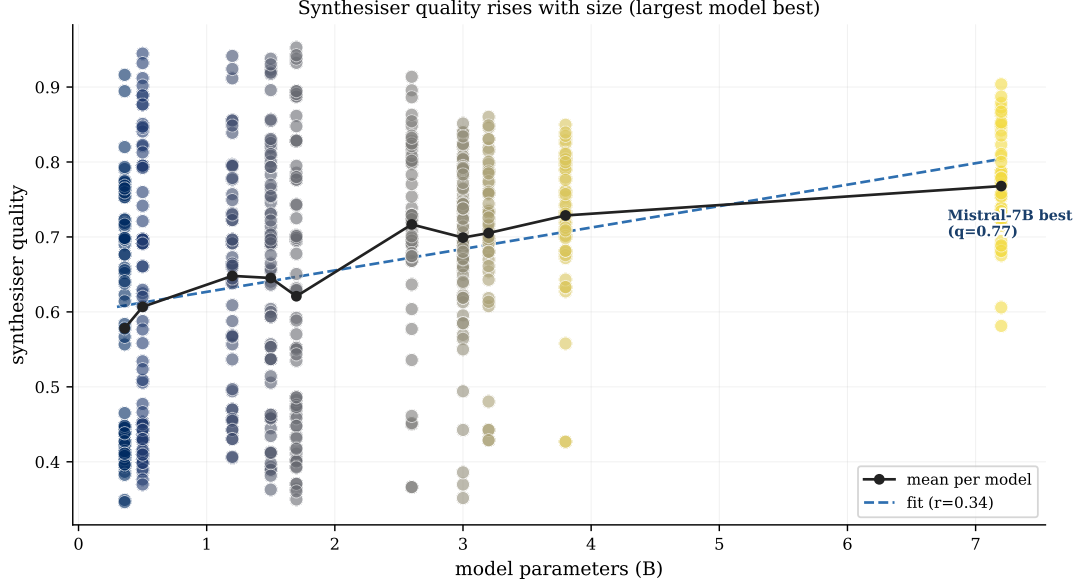


Figure 19: Synthesiser quality versus parameter count (620 records). Quality rises with size (per-model mean in black); the largest model, Mistral-7B, is best. This is the reverse of the specialists and parallels the dispatcher.

Decomposing the synthesiser metric. The aggregate hides a more informative structure (Fig. 20). Of the five components, *context precision* and *context recall* are **constant across all synthesiser models** (0.82 and 0.52): under the oracle-retrieval evaluation (§3.5) the contexts are the gold passages, supplied identically to every model, so these retrieval metrics depend on the context set, not the generative model. The aggregate quality is driven almost entirely by *correctness* (correlation ≈ 1.0 with the aggregate) and *answer relevancy*, both of which rise with model size. Strikingly, *faithfulness moves in the opposite direction*—it **falls** as models grow (correlation -0.70 with quality). The larger synthesisers are less faithful to the supplied context yet more correct overall, indicating they lean on parametric (in-weights) knowledge rather than the provided passages. For a legal assistant this is a double-edged property—higher correctness but weaker grounding—and it is invisible to the aggregate score alone.

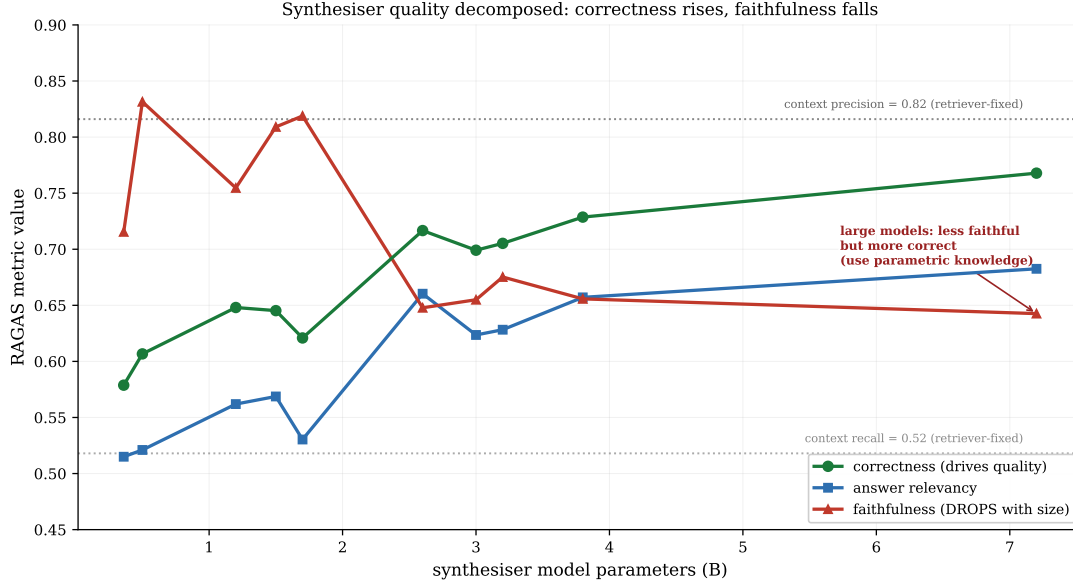


Figure 20: Synthesiser quality decomposed into RAGAS components. Correctness and answer relevancy rise with size; faithfulness *falls* (larger models rely on parametric knowledge); context precision/recall are retriever-fixed constants. The aggregate is correctness-dominated.

8.8 The central finding: three roles, three scaling laws

Putting the three roles together yields the paper’s sharpest allocation insight (Fig. 21). The *dispatcher* (routing) and the *synthesiser* (composition) both improve with model size and are best served by the *largest* models that fit. The *specialists* (domain-grounded RAG) do not: their quality peaks at Llama-3.2-1B and then plateaus, so they are best served by *small* models. The three roles therefore demand models from opposite ends of the size spectrum simultaneously. No single global rule (“largest that fits” or “smallest that fits”) is correct for the pipeline; only a *per-role* allocator can place a large dispatcher and synthesiser alongside small specialists—which is precisely the structure the MILP encodes. This is the measured justification for role-aware allocation, and it is the strongest argument in the paper that the optimization problem is real.

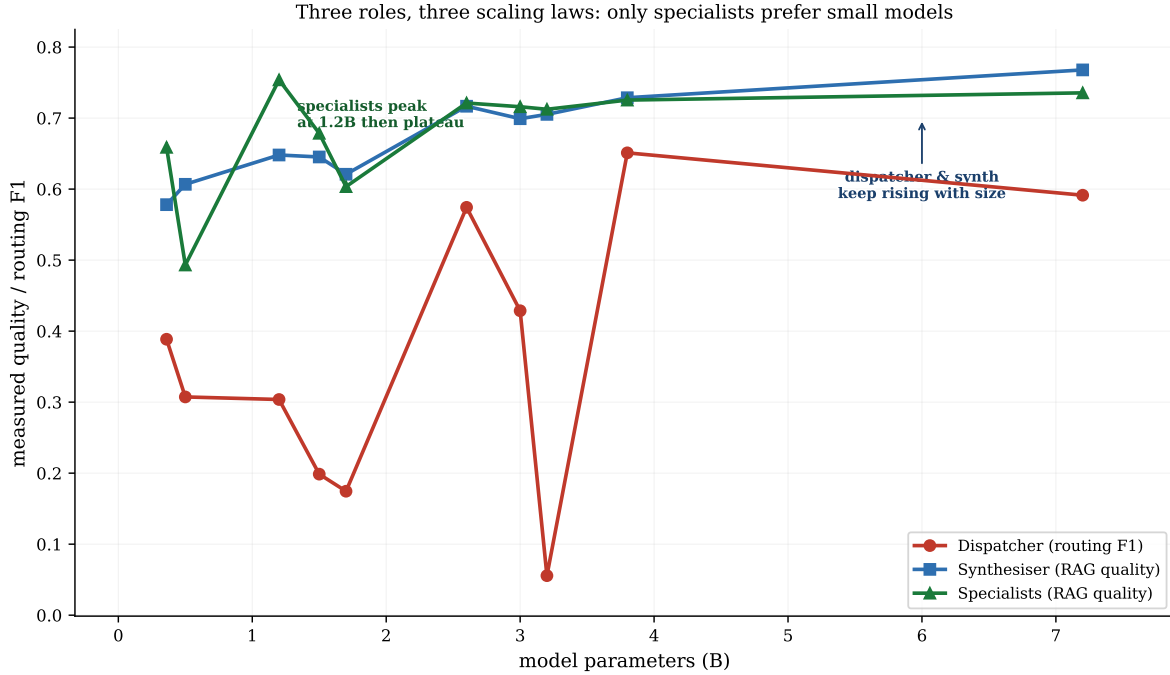


Figure 21: Measured quality versus model size for the three roles. Dispatcher (routing F1) and synthesiser (RAG quality) rise with size; specialists peak at 1.2 B and plateau. The roles prefer opposite ends of the size range, which a per-role allocator can exploit. The dispatcher curve is noisier (e.g. the degenerate Llama-3.2-3B outlier, §8); the trend is nonetheless clear.

8.9 Why three regimes? A mechanistic account

The three scaling laws are not arbitrary; they follow from what each role’s task demands of a model, and the RAGAS decomposition (§8.7) supplies direct evidence. We offer the following account, which turns the empirical finding into a *predictive* hypothesis.

The governing variable is how much the task rewards parametric knowledge versus penalizes it. A model’s quality on a task is, loosely, a sum of two size-dependent effects: larger models bring more parametric knowledge and stronger reasoning (helps), but they also drift further from a short supplied context when left unconstrained (can hurt grounding). The balance differs by role:

- **Dispatcher (routing) — rewards scale.** Routing is short structured classification over the query: no long generation, nothing to hallucinate, and the decision benefits from broad world/linguistic knowledge to disambiguate domains. The penalty term is essentially absent (there is no context to be unfaithful to), so quality is increasing in size. Hence routing F1 rises with parameters.
- **Specialist (domain-grounded RAG) — penalizes scale past a point.** The specialist must answer *from the retrieved legal passage*. Here the two effects oppose: a little more capability helps, but larger models increasingly substitute parametric (and possibly wrong or generic) knowledge for the supplied article, which on narrow legal questions is a net negative. The measured signature is exactly this: for the synthesiser we observe correctness rising while *faithfulness falls* with size (§8.7); the specialists, whose entire job is grounding, hit the crossover earlier—quality peaks at ≈ 1.2 B and plateaus. Small grounded models win because the task rewards fidelity to context over recall of weights.
- **Synthesiser (composition) — rewards scale, but less cleanly.** Composing several

specialist answers into a coherent whole is a longer generative task that benefits from fluency and reasoning (helps, like routing), yet it also shows the falling-faithfulness signature (like the specialists). The first effect dominates over our range, so quality rises with size, but more slowly and less monotonically than routing—which is what the curve shows.

The prediction. This account is falsifiable and portable: a role’s optimal model size should track its *grounding requirement*. Tasks that must be faithful to a short supplied context (extraction, grounded QA, citation) should favour small models and exhibit an early quality peak; tasks that are classification-like or that legitimately draw on parametric knowledge (routing, open-domain synthesis, summarization without a single source of truth) should favour larger models. A direct test is to manipulate the grounding requirement—e.g. shrink or remove the retrieved context—and predict that the specialist’s optimal size *increases* as grounding is weakened, because the penalty term vanishes. We do not run that ablation here; it is the natural next experiment, and the point is that the three-regime finding now comes with a mechanism that says *when* to expect each regime, not merely that we observed three.

8.10 Retriever diagnostic

Separately from the oracle-retrieval quality scores, we evaluate the *live* FAISS retriever (§3.5) against the gold authority ids. Across query–specialist pairs, context recall is frequently low (0.17–0.43 on many queries) and—crucially—*identical for the specialist and the synthesiser on the same query*, because both consume the same retrieved passages. Retrieval quality is therefore a system-level bottleneck independent of which generative models are allocated: no model choice can recover information the retriever failed to surface. This bounds the achievable pipeline quality and is an orthogonal target for future improvement (better chunking, embeddings, or reranking), separate from the allocation problem studied here.

9 Quality-Aware Allocation

This section closes Q2: having shown the proxy is the wrong objective, we put measured quality inside the optimizer and compare. The measured results motivate replacing the parameter proxy with measured quality *inside* the optimizer. We now do so, giving the quality-aware counterpart of the MILP in §6 and the resulting measured frontier. This is the paper’s culminating result: an allocator that maximizes *measured* pipeline quality, and a direct demonstration that it departs from the parameter-proxy optimum.

9.1 Formulation

The structure is identical to §6—three role-slots, load-once memory, serial latency—but the objective is the measured pipeline quality. Let Q_c^{F1} be the dispatcher’s measured routing F1 for configuration c , Q_c^{syn} the synthesiser’s measured quality, and $\bar{Q}_c^{\text{spec}}$ the specialist model’s measured quality averaged over the activated domains. With role-slot assignment variables $x_{d,c}, x_{s,c}, x_{y,c}$ and loaded-config variables z_c as before:

$$\max \quad w_d \sum_c Q_c^{\text{F1}} x_{d,c} + \sum_c \bar{Q}_c^{\text{spec}} x_{s,c} + w_y \sum_c Q_c^{\text{syn}} x_{y,c} \quad (18)$$

subject to one configuration per role ($\sum_c x_{r,c}=1$), load-only-if-used ($x_{r,c} \leq z_c$), one context per weight-sharing group, the load-once memory budget $\sum_c \mu_c z_c \leq M$, and the serial latency chain

$$\sum_c \ell_c^r x_{d,c} + k \sum_c \ell_c^g x_{s,c} + \sum_c \ell_c^g x_{y,c} \leq \varepsilon. \quad (19)$$

We set $w_d=0.15$ (routing weighted below generation) and $w_y=1$; the weights are a modelling choice, stated as such. The program is linear and solves in well under a second.

Scope and assumptions. This objective is *additive* across roles: it sums the three measured qualities and does not model the router-gating interaction (a specialist’s contribution being wasted when the router misroutes), which in the full system is a bilinear term. The specialist quality is the model’s mean over activated domains (the shared-specialist abstraction of §6), and the synthesiser quality is measured against fixed reference specialist answers (§8), so it is independent of the allocation and keeps the program linear. We keep this additive form as the primary allocator for clarity, and then *relax the additivity*—adding the measured bilinear router–specialist gating—in §10, which shows the coupling changes the optimal allocation.

9.2 The measured quality–latency frontier

Solving (18)–(19) over a sweep of ε for each k yields the measured quality–latency frontier (Fig. 22). The optimizer, given only measured quality, *independently recovers the three-regime structure*: at essentially every budget it selects a **large dispatcher (Mistral-7B)**, a **small specialist (Llama-3.2-1B)**, and a **large synthesiser (Mistral-7B)**. It was told nothing about roles preferring different sizes; it reconstructs that pattern purely by maximizing measured quality under the constraints. The frontier plateaus near $Q=1.60$ once the budget admits this mixed allocation.

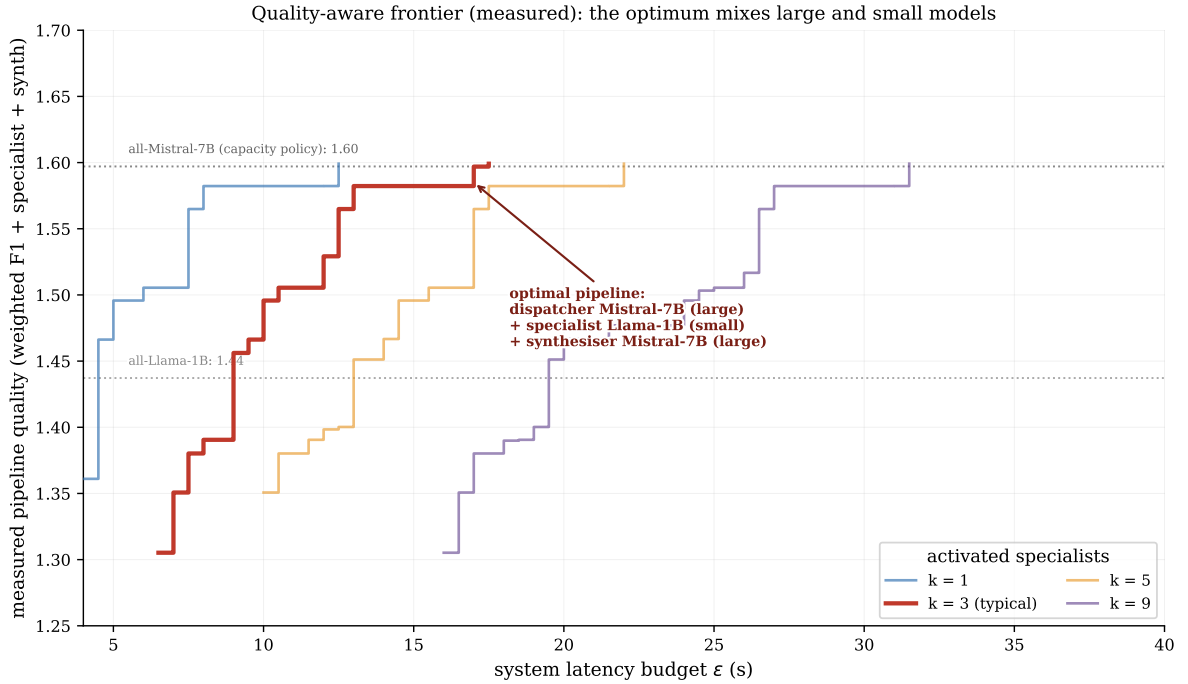


Figure 22: Measured quality–latency frontier from the quality-aware MILP, for $k \in \{1, 3, 5, 9\}$. The optimal allocation mixes model sizes—large dispatcher and synthesiser, small specialist—recovering the three-regime structure from data. Dotted lines mark two fixed-model policies, both dominated.

9.3 Optimization beats fixed policies and the parameter proxy

Two comparisons establish that the quality-aware optimization is necessary, not cosmetic. First, against *fixed-model* policies at $k=3$: the mixed optimum scores $Q=1.60$, against 1.60 for the best single all-Mistral-7B configuration and 1.44 for all-Llama-3.2-1B. We report this honestly: an *unconstrained* all-Mistral policy nearly ties the mixed optimum ($\Delta=0.002$), because Mistral

is also the best routing and synthesis model—but only at the far right of the latency axis, where its full sequential chain is affordable. The decisive comparison is therefore at *matched latency*. Second, and most directly, against the *parameter-proxy* allocator of §7: at matched latency the proxy is sub-optimal at **every one of the 26 budgets tested**, by a mean of 0.13 and up to 0.18 quality points (Fig. 23)—a gap several times larger than the proxy’s gap against a greedy sort (§7). The proxy’s failure is mechanistic: maximizing parameters spends the latency budget on a large dispatcher and is then forced onto small specialist *and* synthesiser models, so its quality can even *decrease* as the budget grows. This is the quantitative core of the paper’s thesis: **a parameter proxy and a measured-quality objective induce materially different optimal configurations**, and only the latter tracks what the system actually delivers.

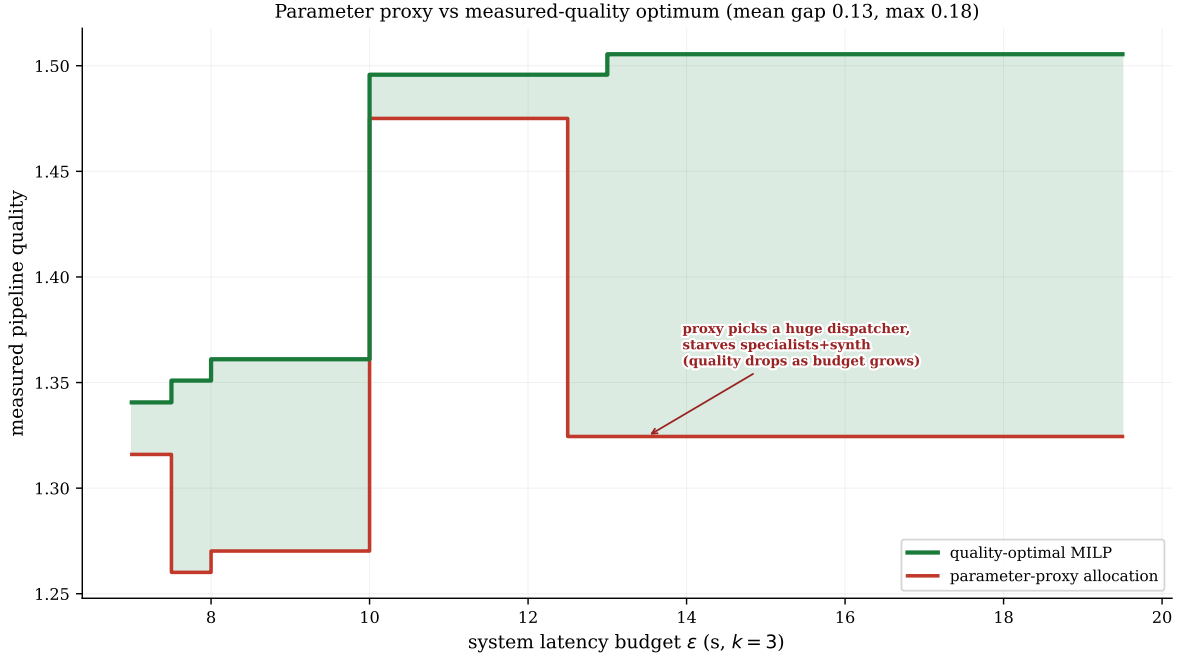


Figure 23: Parameter-proxy allocation versus the measured-quality optimum at matched latency ($k=3$). The proxy is dominated at every budget (mean gap 0.13, max 0.18) and is non-monotone—spending budget on a large dispatcher starves the other roles. Measured quality and the parameter proxy yield different optima.

Robustness to the objective weights. Because w_d and w_y are a modelling choice, we resolve the quality-aware MILP across a grid $w_d \in \{0.05, 0.15, 0.30, 0.50, 1.0\}$, $w_y \in \{0.5, 1.0, 1.5\}$ (at $\epsilon=15$, $k=3$). The qualitative conclusion is invariant: in **all 15 weight combinations the optimal specialist is the small Llama-3.2-1B while the dispatcher and synthesiser remain large** (Fig. 24). The mixed large/small/large structure—and hence the departure from the parameter proxy—is therefore not an artifact of the chosen weights; only the second-order choice between the large models (Phi-3.5 vs Mistral-7B) shifts with the weighting.

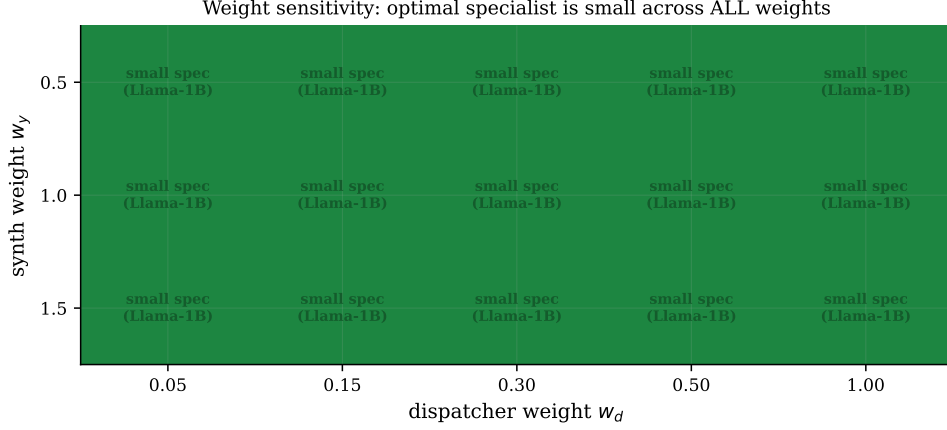


Figure 24: Weight sensitivity of the quality-aware optimum ($\varepsilon=15$, $k=3$). Across the full $w_d \times w_y$ grid the optimal specialist is always the small Llama-3.2-1B (dispatcher and synthesiser stay large), so the qualitative conclusion is weight-robust.

10 Gated Allocation: Modelling Router–Specialist Coupling

This section answers Q3: do the roles interact, and when does modelling that interaction change the answer? The quality-aware objective (18) is *additive*: it sums the three roles’ qualities as if independent. The real pipeline is not. A specialist’s answer is produced *only if the dispatcher routes the query to it*; a misroute wastes the specialist entirely, however good it is. The specialist’s effective contribution is therefore *gated* by the dispatcher’s routing behaviour—a coupling between two allocation decisions. This is the genuinely combinatorial core of the problem, and the reason a per-role sort cannot solve it. The heterogeneous analysis of §7 modelled this coupling only on *synthetic* affinities; here we model it on *measured* data and solve it exactly.

10.1 Measured routing recall

From the dispatcher logs we estimate, for each dispatcher model m and domain δ , the **per-domain routing recall**

$$\rho_{m,\delta} = \frac{\#\{\text{queries with } \delta \in \text{gold set, routed to } \delta \text{ by } m\}}{\#\{\text{queries with } \delta \in \text{gold set}\}} \in [0, 1]. \quad (20)$$

This is distinct from the global routing F1 of §8: F1 averages over *all* domains, whereas $\rho_{m,\delta}$ is the recall on the specific domain that will be activated. The two can diverge sharply—a model can have mediocre global F1 yet excellent recall on a particular subset of domains (Fig. 25), and it is the latter that determines whether an activated specialist is actually reached.

10.2 Gated objective and exact linearization

Let the dispatcher, specialist, and synthesiser model-choice indicators be $y_m^d = \sum_{c: \text{model}(c)=m} x_{d,c}$ (and likewise y_m^s, y_m^y), each $\in \{0, 1\}$ with $\sum_m y_m^d = 1$. For the set $\mathcal{A}$ of activated domains, the **gated pipeline quality** is

$$Q_{\text{gated}} = w_d Q_d^{\text{F1}} + \frac{1}{|\mathcal{A}|} \sum_{\delta \in \mathcal{A}} \underbrace{\left(\sum_m \rho_{m,\delta} y_m^d \right)}_{\text{router reaches } \delta} \cdot \underbrace{\left(\sum_{m'} Q_{m',\delta}^{\text{spec}} y_{m'}^s \right)}_{\text{specialist quality on } \delta} + w_y Q_y^{\text{syn}}. \quad (21)$$

The middle term is **bilinear**: it multiplies dispatcher indicators by specialist indicators. We linearize it *exactly* (binary $\times$ binary) with McCormick envelopes. For each activated domain δ and model pair (m, m') with $\rho_{m,\delta} > 0$ and $Q_{m',\delta}^{\text{spec}} > 0$, introduce $w_{\delta,m,m'} \in [0, 1]$ with

$$w_{\delta,m,m'} \leq y_m^d, \quad w_{\delta,m,m'} \leq y_{m'}^s, \quad w_{\delta,m,m'} \geq y_m^d + y_{m'}^s - 1, \quad (22)$$

which force $w_{\delta,m,m'} = y_m^d y_{m'}^s$ at any binary solution. The gated term becomes the linear $\sum_{\delta,m,m'} \rho_{m,\delta} Q_{m',\delta}^{\text{spec}} w_{\delta,m,m'} / |\mathcal{A}|$, so (21) is a MILP under the same memory and latency constraints ((19)) and solves with CBC. Applying the envelopes at the *model* level (not configuration) keeps the auxiliary-variable count at $|\mathcal{A}| \cdot |\mathcal{M}_d| \cdot |\mathcal{M}_s| \approx 3 \times 11 \times 11$, well within reach; a configuration-level encoding would be $\sim 40\times$ larger and is unnecessary because ρ and Q^{spec} depend on the model, not the quantization.

10.3 Results: the coupling reshapes the optimum

Allocation. Solved over the activated domains (*succ. legitima/testamentaria, tutela minori*) at $k=3$, the gated MILP selects a *different* allocation from the additive one at every budget. The change is concentrated, tellingly, in the **dispatcher**: the additive model picks a high-global-F1 router (Phi-3.5, or Mistral-7B at large budgets), whereas the gated model picks **SmolLM2-360M**—a router with low global F1 (0.39) but near-perfect recall (0.99) on the activated domains (Fig. 25). The gated objective values recall *on the domains actually used*, and the tiny router additionally frees memory and latency for the other roles. The specialist stays small (Llama-3.2-1B), consistent with the three-regime finding: the coupling reshapes the *dispatcher* decision, exactly where the interaction lives.

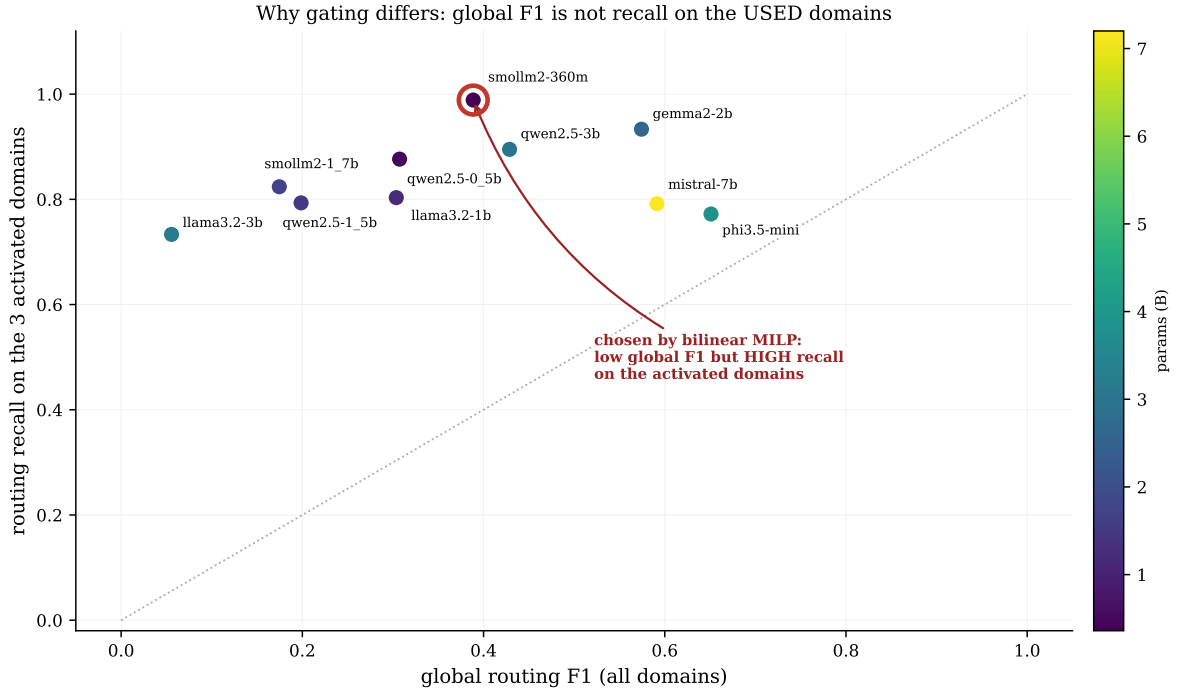


Figure 25: Why gating changes the dispatcher choice. Each model’s routing recall on the three activated domains (y) exceeds its global F1 (x); that gap is what the additive objective misses. SmolLM2-360M (circled) has low global F1 but the highest activated-domain recall, so the gated MILP selects it. Colour is model size.

Frontier. Sweeping ε traces the gated quality–latency frontier (Fig. 26) for $k \in \{1, 3, 5\}$. It is

the measured, gated analogue of the capacity frontier of §7: a staircase rising to a plateau, but now in *true pipeline quality* accounting for routing.

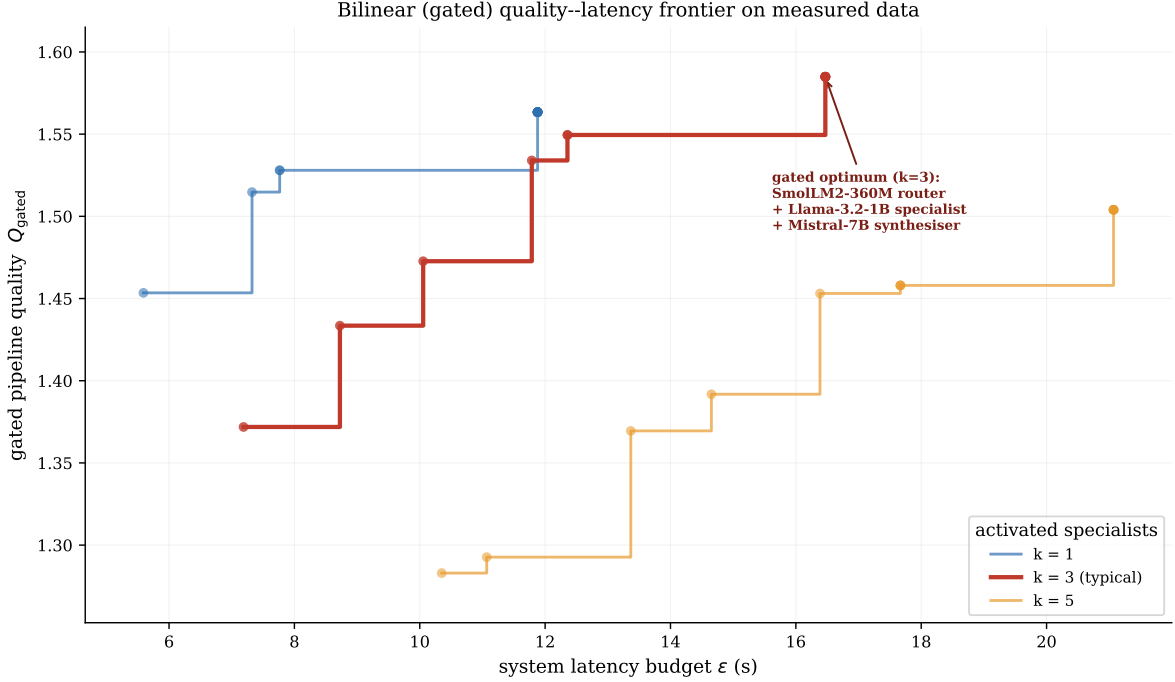


Figure 26: Gated quality–latency frontier from the bilinear MILP on measured data, $k \in \{1, 3, 5\}$. Quality is Q_{gated} from (21); the annotated $k=3$ optimum mixes a high-recall small router, a small specialist, and a large synthesiser.

Fair comparison. To quantify the cost of ignoring the coupling, we take the *additive* optimum at each budget and re-score it under the *true* gating (21)—i.e. we ask how the allocation that an additivity-blind optimizer would deploy actually performs once the real router gates it. The gated optimum dominates it at **42 of 44 budgets across** $k \in \{1, 3, 5\}$, by a mean of 0.18 and up to 0.32 gated-quality points (Fig. 27). The additive optimizer systematically over-values high-global-F1 routers that miss on the activated domains, and under-values the small high-recall router the gated model prefers.

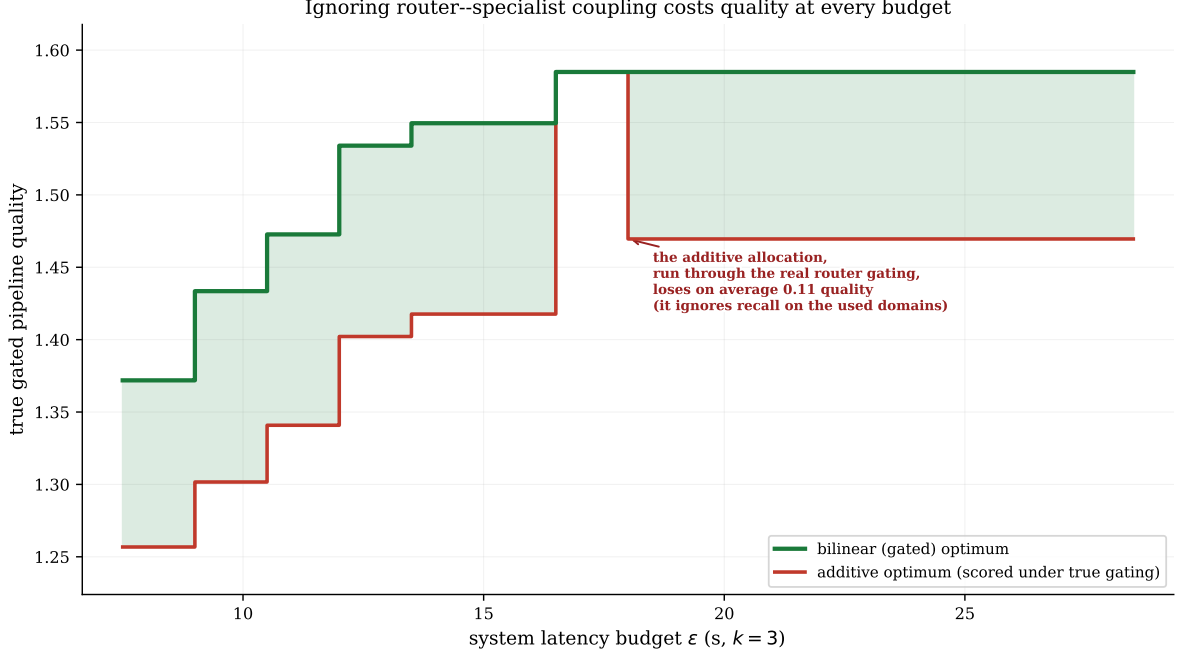


Figure 27: Cost of ignoring router-specialist coupling ($k=3$). Both allocations are scored under the *true* gating: the bilinear optimum (green) versus the additive optimum (red) re-scored with (21). The additive allocation loses quality at every budget; the shaded region is the forgone quality.

This closes the gap between the paper’s hard formulation and its measured evidence: on real data, modelling the router-specialist coupling yields a materially different and better allocation than the additive objective, and it does so within an exactly-linearized MILP. **Caveat:** $\rho_{m,\delta}$ (20) is estimated from few queries per domain (§ Limitations), so the *specific* winning router is indicative; the *structural* results—that coupling changes the optimum, that recall-on-activated-domains (not global F1) is the right quantity, and that the gated optimum dominates the additive one under true gating—are the robust claims.

10.4 When is optimization worth it? A coupling sweep

The baseline of §7 (greedy ties the proxy on 92% of points) and the gated result above (the MILP wins by 0.18) are the two ends of a single axis: how strongly the roles are *coupled*. We make this precise. Introduce a coupling strength $\lambda \in [0, 1]$ that interpolates the specialist’s effective routing recall,

$$\rho_{m,\delta}^{\text{eff}}(\lambda) = (1 - \lambda) \cdot 1 + \lambda \cdot \rho_{m,\delta}, \quad (23)$$

so $\lambda=0$ is the fully decoupled (additive) system—every activated specialist is always reached—and $\lambda=1$ is the full measured gating. At each λ we compute both the gated-MILP optimum and a natural greedy heuristic (each role chosen by its own quality within its latency share), and score *both* under the same $\rho^{\text{eff}}(\lambda)$ for a fair comparison.

The result (Fig. 28) is a clean, near-linear law: the value of optimization grows monotonically with coupling, from a mean MILP-greedy gap of 0.06 at $\lambda=0$ to 0.17 at $\lambda=1$ (slope ≈ 0.11). Two readings matter. First, at $\lambda=0$ the gap is small but *not* zero (0.06): even with no routing coupling, the shared-memory and shared-latency constraints couple the roles enough that a greedy per-role choice is slightly sub-optimal. Second, the gap rises steadily as gating tightens, so **the more a multi-agent system gates downstream agents on upstream decisions, the more an exact allocator is worth over a heuristic**. This turns the paper’s specific result into a general statement: optimization is marginal for loosely-coupled pipelines and

increasingly essential for tightly-coupled ones, with the crossover quantified on real data. A practitioner can read off, from an estimate of their system’s coupling, whether the MILP is worth deploying.

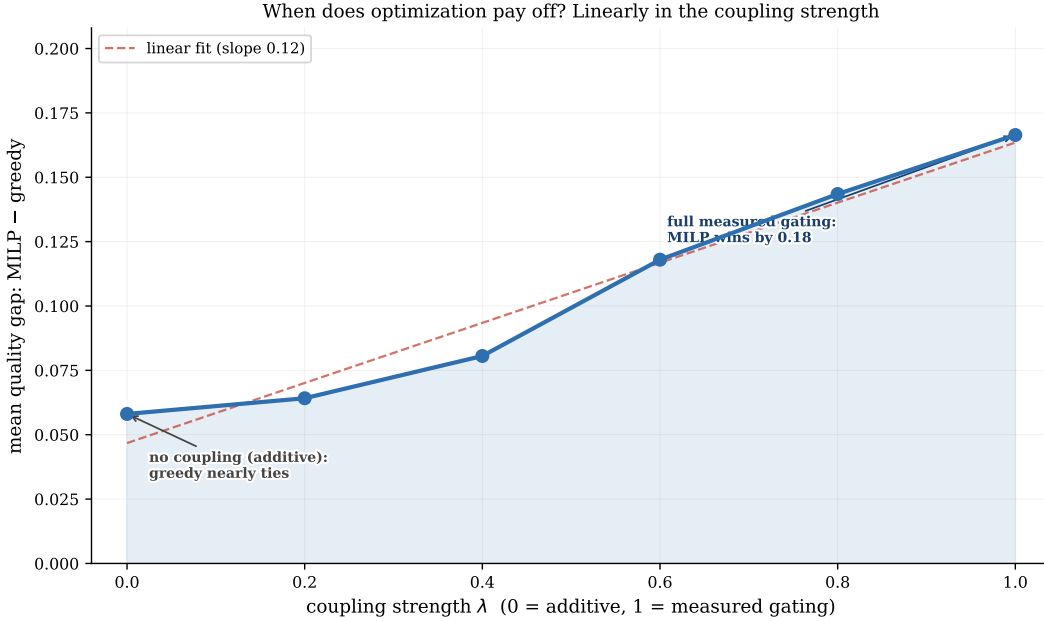


Figure 28: When optimization pays off. Mean quality gap between the gated MILP and a greedy per-role heuristic, as a function of coupling strength λ ((23); 0 = additive, 1 = full measured gating), $k=3$. The gap grows near-linearly (slope ≈ 0.11): optimization is marginal when roles decouple and increasingly valuable as gating tightens.

10.5 Robustness to execution mode: the batched extreme and an exact concurrent MILP

Our primary latency model (§6) treats the activated specialists as running *serially* on the shared GPU, $L_{\text{sys}} = L_d + k L_s + L_y$ —the conservative worst case, since several specialists sharing one loaded model cannot run truly in parallel on one instance. The opposite extreme is full *batching*, in which the activated specialists are issued as a single forward pass and the stage cost is one generation, $L_d + L_s + L_y$, independent of k . A real runtime sits between these two. We treat the serial model as primary (it is the conservative accounting, consistent with load-once memory, and the basis of the frontiers above) and use this section to confirm the conclusions survive the batched extreme; the exact batched model is what the released solver’s `worst_case` mode optimizes.

We introduce a *serialization factor* $\sigma \in [\frac{1}{k}, 1]$ that interpolates the specialist latency term,

$$L_{\text{sys}}(k, \sigma) = L_d + (1 + \sigma(k - 1)) L_s + L_y, \quad (24)$$

so $\sigma=1$ is the serial model used as primary and $\sigma=\frac{1}{k}$ is ideal batching (the k homogeneous specialists cost as much as one call). σ enters *only* the latency constraint; the load-once memory bound and all quality terms are unchanged, since a shared specialist occupies its memory once regardless of execution terms. We re-solve the quality-aware MILP at $k=3$ across the ε -grid for $\sigma \in \{1, 0.66, 0.5, \frac{1}{3}\}$ (Fig. 30).

Result. Batching shifts the frontier *left*—the same quality at lower latency—but does not change its shape or the optimal allocation. The *structural* optimum is invariant: the dispatcher and synthesiser remain large and the specialist remains small (Llama-3.2-1B) in 100% of budgets

for $\sigma \in \{1, 0.66\}$, in 96% at $\sigma=0.5$ (a single large-specialist pick at the grid extreme $\varepsilon=30$ s), and in 84% even at ideal batching $\sigma=\frac{1}{3}$. Crucially, across the realistic on-device regime $\varepsilon \leq 20$ s the specialist is small at 100% **of budgets in all four execution modes**. The choice changes only at large budgets ($\varepsilon \geq 27$ s) under strong batching ($\sigma \leq 0.5$), where a large specialist (Mistral-7B) becomes affordable—which is exactly the expected behaviour: when batching makes the specialist term cheap and the budget is generous, the optimizer can spend it on a large specialist without violating the constraint. This does not weaken the three-regime finding; it confirms its mechanism (the specialist is small *because* of the latency cost of repeated calls, and relaxing that cost is precisely what would admit a larger one).

An exact concurrent MILP (the $\sigma=\frac{1}{k}$ endpoint, without interpolation). The serialization sweep *estimates* batching by scaling the latency term; we can instead model the batched endpoint *exactly* inside the program. Since the activated specialists share one loaded configuration, a single batched pass costs the latency of that one configuration. Introduce a continuous auxiliary $\Lambda \geq 0$ and a cover constraint, replacing the sequential chain by

$$L_{\text{sys}}^{\text{conc}} = L_d + \Lambda + L_y \leq \varepsilon, \quad (25)$$

$$\Lambda \geq \ell_c^g x_{s,c} \quad \forall c. \quad (26)$$

Because exactly one $x_{s,c}=1$, Λ collapses to the chosen specialist’s generation latency, so $L_{\text{sys}}^{\text{conc}} = L_d + L_s + L_y$: this is the $\sigma=\frac{1}{k}$ endpoint of (24), expressed as one linear constraint rather than a parametric scan. The linearization is exact (one auxiliary plus $|\mathcal{K}_s|$ rows) and the program stays a sub-second MILP. (Both (24) and (25) use the single-stream performance table, so each is a *formulation* of batching rather than a measured batched-throughput campaign; the two agree, and a direct measurement is left to future work.)

Comparison. Re-solving the full ten-model quality-aware MILP under the sequential chain versus the exact concurrent chain (Fig. 29) makes the $\sigma=\frac{1}{k}$ endpoint exact rather than interpolated; it is the same endpoint as the sweep, not an independent confirmation, but expressing it as a hard constraint lets us read the feasibility boundary directly. Concurrent serving strictly enlarges the feasible region: the pipeline becomes feasible from $\varepsilon=4$ s (versus 7 s sequentially) and attains higher quality at every matched budget where both are feasible. The *allocation*, however, is invariant: the optimal specialist is a sub-1.3B model at 100% of feasible budgets under both chains: it is the small Llama-3.2-1B everywhere (13/14 concurrent, 9/11 sequential) except the very tightest budgets ($\varepsilon=4$ s concurrent; $\varepsilon \leq 8$ s sequential), where only the even smaller SmolLM2-360M fits the slot. The dispatcher and synthesiser grow with the budget identically, and both frontiers plateau at the same $Q \approx 1.60$. The estimate and the exact formulation thus agree—the execution model moves the frontier but leaves the three-regime optimum untouched.

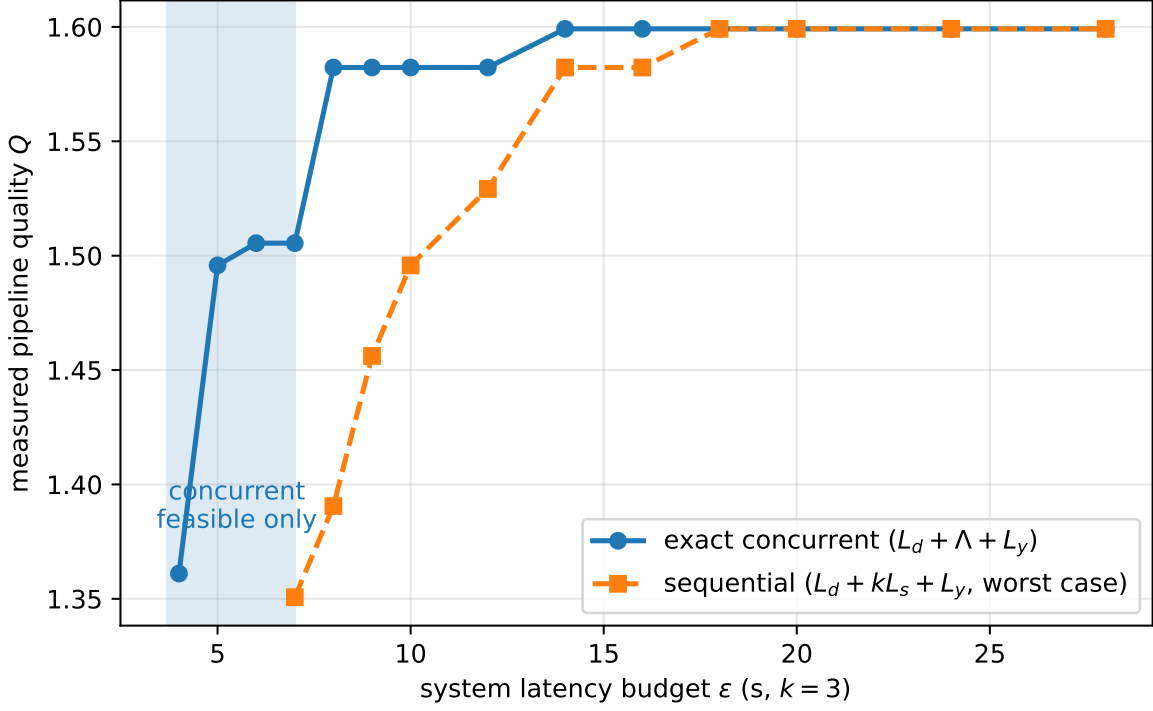


Figure 29: Quality-latency frontier under the sequential chain ($L_d + kL_s + L_y$) versus the exact concurrent chain (Eqs. 25–26), full pool, $k=3$. Concurrent serving shifts the frontier left (feasible from 4 s versus 7 s, higher Q at matched ϵ) but the optimal allocation—small Llama-3.2-1B specialist, large dispatcher and synthesiser—is unchanged across the entire feasible range, matching the σ -sweep of Fig. 30.

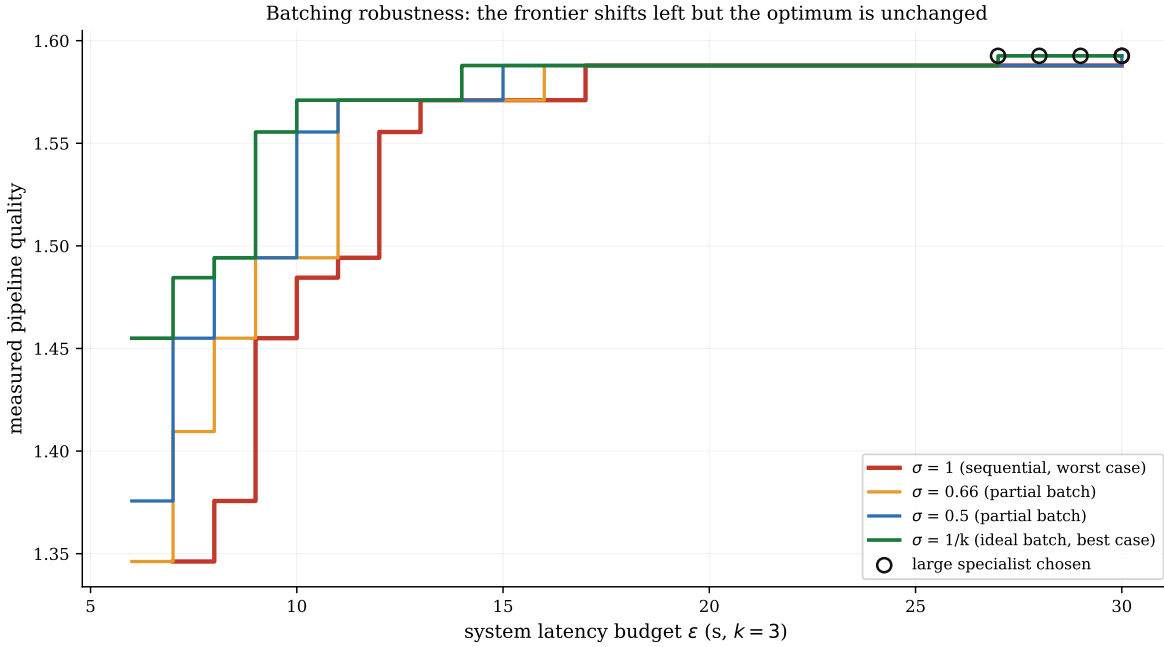


Figure 30: Robustness to execution mode ($k=3$). Quality-latency frontier under the serialization factor σ (24): $\sigma=1$ (sequential, worst case) to $\sigma=1/k$ (ideal batching, best case). Batching shifts the frontier left (lower latency for the same quality) but the small-specialist optimum is unchanged across the realistic budget range.

11 Generality of the Allocation Results

This section answers **Q4**: which of the paper’s modelling choices are load-bearing? We isolate the two most consequential—the heterogeneous model pool and the execution model—and test each as a controlled, single-variable change against the same quality-aware MILP (§9), the same memory budget $M=6.99$ GB, and the same measured quality tables. The execution-model half is treated in §10.5 (the optimum is invariant under the exact concurrent MILP); here we test the pool, and find that *two* distinct findings of the paper—the non-monotone three-regime structure (§8) and the value of modelling router–specialist coupling (§10)—both depend on it. The picture that emerges is clean: what is load-bearing for the paper’s quality conclusions is the diversity of the candidate pool, not the latency accounting—the two pool-dependent findings collapse in unison within a single dense family, while the execution model leaves the optimum untouched.

11.1 Single-family ablation: heterogeneity creates the three regimes

Setup. We restrict the pool from ten heterogeneous models to a single dense family—Qwen2.5 in three sizes (0.5/1.5/3 B), three quantizations, and three contexts, i.e. $3 \times 3 \times 3 = 27$ configurations (18 of which satisfy the specialist’s minimum context $c_{\min} \geq 4096$; all fit M). Every configuration carries the same measured quality used in §8, so the only changed variable is the *diversity* of the pool. The objective and the load-once memory accounting are identical to §9; the sweep is solved under the exact concurrent chain of §10.5 with the agents’ minimum-context constraints enforced—the same execution model as the allocator of §12.4—so the single-family and full-pool analyses are compared like-for-like on the realistic execution mode.

No single family reproduces the full-pool optimum. The headline finding (§8) is that specialist quality is *non-monotone* in size and peaks at Llama-3.2-1B (mean quality 0.754), beating Mistral-7B (0.736) at $6\times$ fewer parameters (pooled per-record means, the convention of the §8 confidence intervals; the per-configuration scorecard means, 0.74/0.72, rank the models identically). That structure is a property of *cross-family heterogeneity*, and it is not reproduced inside any single family (Fig. 31). The three families with more than one size behave inconsistently and *none* of them recovers the full-pool picture: within Qwen2.5 quality is monotone *increasing* ($0.493 \rightarrow 0.679 \rightarrow 0.716$ for $0.5 \rightarrow 1.5 \rightarrow 3$ B), whereas within both SmolLM2 ($0.659 \rightarrow 0.603$ for $0.36 \rightarrow 1.7$ B) and Llama-3.2 ($0.754 \rightarrow 0.712$ for $1.2 \rightarrow 3.2$ B) it is monotone *decreasing*. The model that creates the full-pool peak—Llama-3.2-1B—wins not because small is better within its own family in some universal sense, but because the small member of *one* family beats the large members of *all* others: the effect lives in the spread *across* families at a fixed size, not in a within-family law. A practitioner who restricts the pool to a single dense family therefore sees an idiosyncratic, family-specific size–quality relation (rising for Qwen, falling for SmolLM2 and Llama) and never the non-monotone three-regime structure that justifies role-aware allocation.

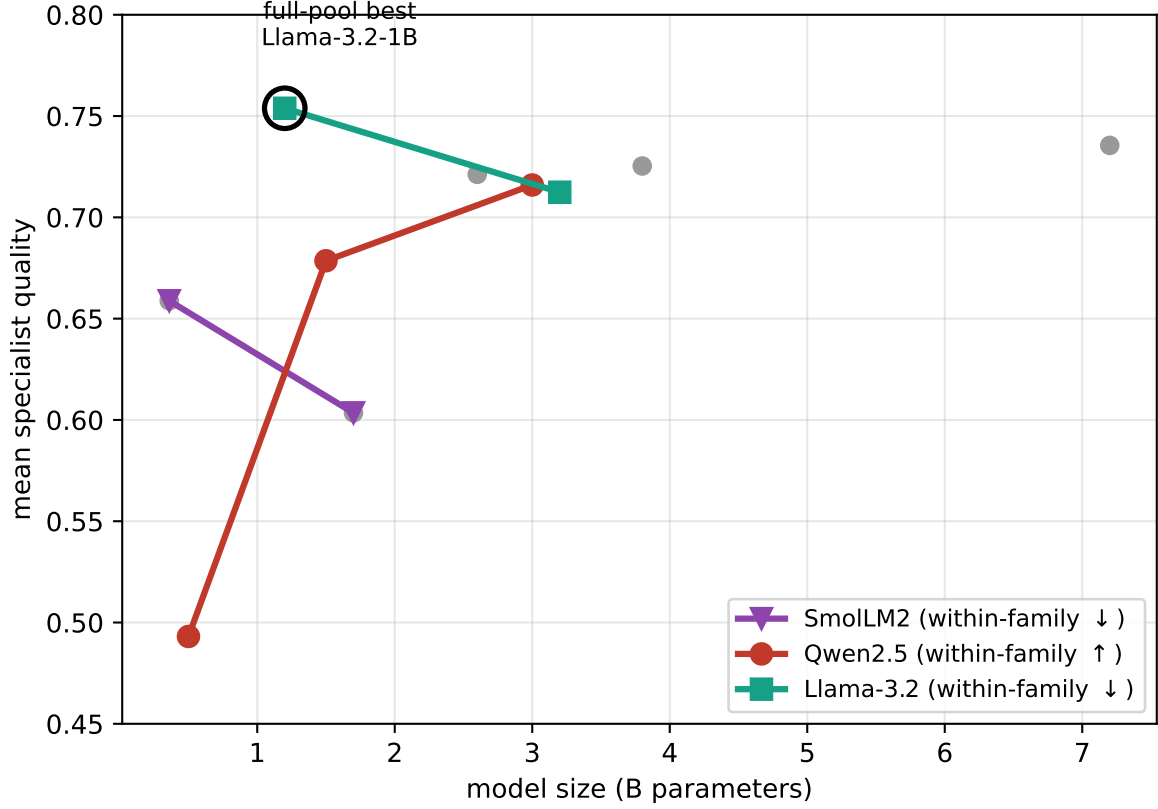


Figure 31: Measured specialist quality versus model size. Across all ten models (grey) the relation is non-monotone and peaks at Llama-3.2-1B (circled, 1.2 B), exceeding Mistral-7B at a fraction of the size. The three families with more than one size disagree on the within-family trend—Qwen2.5 rises with size, while SmolLM2 and Llama-3.2 fall—and none reproduces the full-pool non-monotonicity. The effect is a cross-family one: the small member of one family (Llama-3.2-1B) beats the large members of all others.

The small specialist becomes a constraint artifact. Solving the quality-aware MILP on the Qwen pool, the optimal specialist is the small 1.5 B model at tight latency budgets—but for a reason unrelated to quality. Sweeping ε (Fig. 32), the specialist is 1.5 B for $\varepsilon < 11$ s and then *switches to the largest 3 B model at $\varepsilon=11$ s*, the moment the budget admits it. The 1.5 B choice is driven entirely by the latency of the specialist on the critical path together with the load-once memory budget—sharing the 1.5 B group between specialist and synthesiser frees memory for a large dispatcher—and it disappears as soon as the budget relaxes. This is the opposite of the full-pool behaviour, where Llama-3.2-1B is the optimal specialist at *every* budget (Table 4) because it is the genuine quality maximiser. In the full pool the small specialist wins *on quality*; in the single family it appears only *under a tight constraint*.

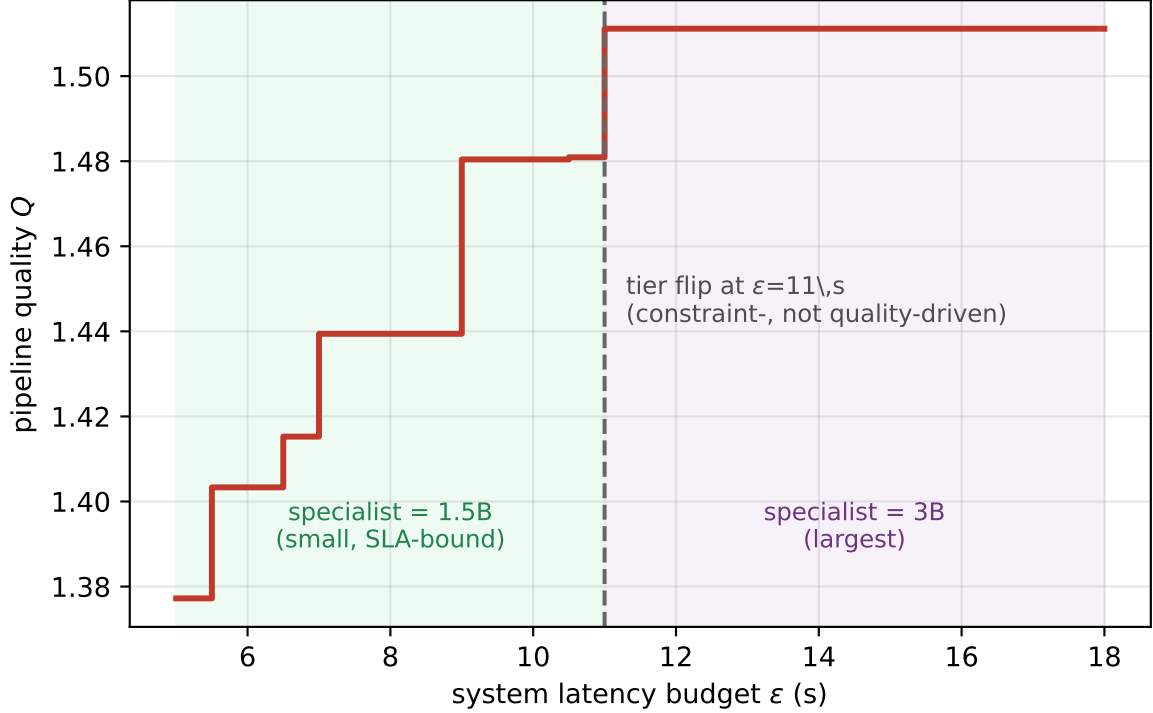


Figure 32: Single-family (Qwen2.5) quality–latency frontier under the quality-aware MILP ($k=3$). The optimal specialist is the small 1.5 B model only while the budget is tight; it switches to the largest (3 B) model at $\varepsilon=11$ s. The small specialist here is a constraint artifact, not the quality-driven choice that Llama-3.2-1B represents in the full pool.

Table 4: Quality-aware optimum, full pool versus single family ($k=3$). In the full pool the small specialist (Llama-3.2-1B) is optimal at *every* budget on quality from $\varepsilon=5$ s (at the single tightest budget only SmolLM2-360M fits the slot, §10.5); in the Qwen pool the small specialist is budget-bound and yields to the largest model once ε grows. Dispatcher and synthesiser are large in both regimes (§9).

ε (s)	pool	optimal specialist
≥ 5	full (10 models)	Llama-3.2-1B (quality-optimal)
< 11	Qwen (1 family)	Qwen2.5-1.5B (budget-bound)
≥ 11	Qwen (1 family)	Qwen2.5-3B (largest)

What the ablation establishes. The role-aware allocation *machinery* is general—it runs unchanged on either pool, and the large-dispatcher/large-synthesiser preference persists in both. But the paper’s sharpest empirical claim, that specialists prefer small models on quality, is a consequence of having a *diverse* pool that contains a small model genuinely strong at grounded generation. A practitioner who restricts to one family will not observe the three-regime effect and will, under a tight budget, reach a small specialist for the wrong reason. This bounds the external validity of the finding and reinforces its mechanism: heterogeneity is the source of the non-monotonicity, and the optimizer is valuable precisely because it exploits cross-family structure a single-family sort cannot see.

11.2 The coupling result is also cross-family

The single-family ablation tests the *capability* finding (§8); we now apply the same controlled restriction to the *coupling* finding (§10), to check whether the value of modelling router–specialist

gating is itself an artifact of pool diversity. We re-solve the gated bilinear MILP and its coupling-blind additive counterpart on the Qwen-only pool, scoring both under the true gating (21).

The result mirrors the capability case exactly. In the full pool the gated optimum selects a *different* dispatcher from the additive one at essentially every budget—the small, high-recall SmolLM2-360M instead of the high-global-F1 Phi-3.5 or Mistral-7B—and dominates it by up to 0.32 gated-quality points. Restricted to Qwen2.5, that gap vanishes: the gated and additive optima select the *identical* dispatcher and specialist at every feasible budget (33/33 integer budgets, $\varepsilon = 8\text{--}40\text{s}$, under the sequential chain of §10; 37/37 under the concurrent chain), and the gated-quality gap is zero. The reason is visible in the recall data: within Qwen the model with the best global F1 (Qwen2.5-3B, 0.43) is *also* the model with the best recall on the activated domains (0.89), so the global-F1 proxy and the activated-domain recall agree and the coupling has nothing to exploit. In the full pool they disagree sharply—SmolLM2-360M has the lowest global F1 (0.39) but near-perfect activated-domain recall—which is exactly the gap the bilinear term captures.

A unified message. Both of the paper’s quality findings draw their force from the same source. The non-monotone three-regime structure (§11.1) and the value of modelling router–specialist coupling (this section) are *both* cross-family phenomena: each collapses in a homogeneous pool and re-appears only when the candidate set spans families with genuinely different size–quality and routing profiles. The execution model, by contrast, leaves the optimum untouched (§10.5). What is load-bearing for this paper’s conclusions is therefore the *diversity of the candidate pool*, not the latency accounting—and that diversity is the realistic default for on-device multi-model deployment, where the whole point is to mix small specialised models with a larger router and synthesiser.

Scope of the two collapses. The two results rest on different amounts of evidence, and we state which. The capability collapse is visible across *all three* multi-size families (Qwen2.5, SmolLM2, Llama-3.2; Fig. 31), none of which reproduces the full-pool non-monotonicity. The coupling collapse, by contrast, is demonstrated on the *single* family (Qwen2.5) that has both enough sizes and enough per-domain routing instances to support the gated bilinear MILP; the other two multi-size families are too small on the routing side to re-fit the coupling cleanly. We therefore report the coupling collapse as a single-family result—indicative of the same cross-family mechanism, but not a multi-family law.

12 Dynamic Per-Query Allocation

This section answers Q5. Every allocation so far is *static*: the model in each role is fixed before any query arrives, and each query pays for the same set of activated specialists. Yet most queries need only a subset of the nine specialists—a custody question needs `tutela.minori`, not the succession or matrimonial-property domains. Activating all of them wastes compute, and activating the *wrong* ones wastes compute and injects distractor context into the synthesiser. We therefore add a *dynamic* layer on top of the static allocation that decides, per query, which specialist domains to run—making the activation count $k(q)$ endogenous—while changing nothing about which model serves each role. The static allocator (§9–10) chooses the substrate; the dynamic layer routes over it.

Evaluation data and its provenance. The static-allocation results of §8–10 use the 25 gold-annotated queries of §3, whose gold supporting-authority *passages* are required by the oracle-retrieval quality protocol. The dynamic gate needs a different kind of supervision—gold *routing* labels (which specialists each query should activate), not gold passages—so for it we use an expanded routing-annotated set, disjoint in purpose from the quality campaign and

described here for transparency. Three sources, mutually disjoint, are used and must not be conflated: (i) the original 25 gold-*passage* queries supply the per-domain corpus *profiles* (the relevance signal is similarity to these gold passage texts); (ii) a larger set of 94 routing-annotated queries (`expected_agents` labels only, no gold passages) is used solely to *calibrate* the gate’s thresholds; and (iii) a held-out *test* set of 64 queries (49 in-domain across the nine domains, 15 deliberately out-of-scope) is used only to *evaluate*. The 94-query calibration set and the 64-query test set have zero identifier overlap (verified), and neither overlaps the 25-query profile set; so threshold-fitting, profiling, and testing are performed on disjoint data and the evaluation involves no leakage. We stress that the in-domain metric below is *routing* accuracy against gold `expected_agents`, a proxy for answer quality, not reference-checked answer correctness; §13 states this bound precisely.

12.1 An LLM-free per-query gate

For a query q a candidate set $C(q)$ of specialist domains is proposed—by the deployed dispatcher, or, in the variant studied here, by a retrieval router—and a calibrated gate prunes $C(q)$ to the selected set $S(q) \subseteq C(q)$, so $k(q) = |S(q)|$. The gate uses one signal: the similarity between q and each domain’s corpus profile, profiles built offline from the 25-query gold passage texts (§12), embedded with the same `bge-m3` encoder used throughout. Crucially the gate loads *no* second language model—it is a calibrated function of an embedding similarity—so it adds nothing to the load-once memory budget M that governs the static allocation, and it is therefore free in exactly the resource the rest of the paper optimizes.

Let $\rho_d(q) \in [0, 1]$ be the calibrated relevance of domain d to q , obtained by isotonic regression of the raw fused similarity onto gold relevance labels. Domain d is admitted to $S(q)$ when $\rho_d(q) \geq \tau_d$. The per-domain thresholds $\{\tau_d\}$ are *learned from the 94-query calibration set*, not hand-set, by maximizing a recall-favoring F_β criterion of the kept set against the gold specialist set,

$$\tau_d = \arg \max_{\tau} F_\beta(\{\rho_d(q) \geq \tau\}, \text{gold}_d), \quad \beta \geq 1, \quad (27)$$

subject to three robustness controls that matter in the small-calibration regime: a *recall floor* forbidding any τ that drops more than a fixed fraction of a domain’s calibration positives (this rules out the degenerate “threshold so high that nothing passes” solution); a hard ceiling $\tau \leq \bar{\tau}$; and empirical-Bayes *shrinkage* of each per-domain τ_d toward the global threshold with weight proportional to the domain’s sample count, so thin-sample domains borrow strength from the pool. We take $\beta = 3$ (recall-favoring), the appropriate bias for a legal assistant where dropping a relevant specialist is costlier than running an extra one. When no candidate clears its threshold, $S(q) = \emptyset$ and the pipeline abstains—the desired response to an out-of-scope query.

12.2 Routing and latency results

We report routing precision, recall, and F_1 against the gold specialist set, and latency from the measured performance table (Eq. 6) via the selected $k(q)$ —the same axis as the static frontiers of §7 and §9.

Main result. Table 5 and Fig. 33 compare static- k routing (the candidate set run in full) against dynamic gating at $\beta = 3$, across three static allocations: the quality-aware optimum (§9), the gated optimum (§10), and a single-family Qwen allocation. Static routing has high recall (0.93) but low precision (0.24): the router proposes about six of nine domains and most are wrong. The gate roughly doubles precision (to 0.43) while conceding seven points of recall (to 0.86), raising F_1 from 0.33 to 0.50 (+0.17) and cutting mean activation from 6.0 to 3.61. The

routing metrics are *identical* across the three allocations—the gate is allocation-independent, and the allocations separate only on latency (qwen 9.69 s < gated 10.76 s < qa 11.07 s). The routing gain therefore transfers across allocations while cost does not. (We note that this F_1 baseline of 0.33 is the candidate set proposed by the LLM-free retrieval router and is a distinct quantity from the per-model dispatcher routing F_1 of §8, which reaches 0.68; the two are not directly comparable, as the former scores a multi-domain candidate set and the latter a single model’s full routing decision.)

Table 5: Static- k versus dynamic routing on the 64-query held-out test set ($\beta = 3$, per-domain F_β thresholds, **bge-m3**). Latency from the measured performance table under the concurrent model. Routing metrics are identical across allocations (the gate is allocation-independent); the allocations differ only in latency.

allocation	routing	F_1	prec.	rec.	k	lat. (s)
qa-optimum	static	0.33	0.24	0.93	6.00	12.18
	dynamic	0.50	0.43	0.86	3.61	11.07
gated-optimum	static	0.33	0.24	0.93	6.00	11.87
	dynamic	0.50	0.43	0.86	3.61	10.76
qwen	static	0.33	0.24	0.93	6.00	10.67
	dynamic	0.50	0.43	0.86	3.61	9.69

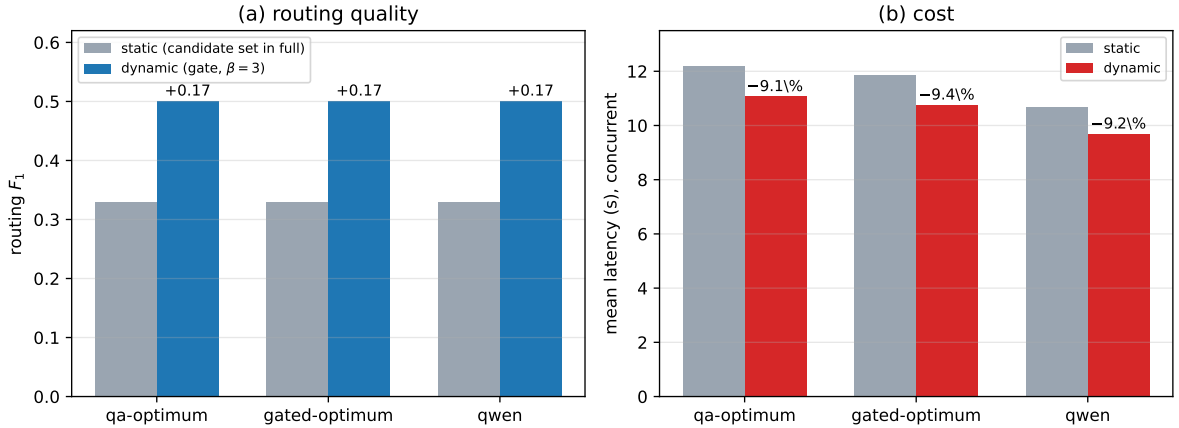


Figure 33: Dynamic routing improves F_1 by 0.17 at every allocation (a) while cutting latency by $\approx 9\%$ under the concurrent model (b). The routing gain is allocation-independent; only latency differs across allocations.

The precision–recall knob and the latency model. Sweeping the F_β parameter (Fig. 34) traces the operating curve. As β rises from 1 to 5, recall climbs from 0.80 to 0.86 and mean k from 3.12 to 3.61, with F_1 nearly flat (0.49–0.50); $\beta = 3$ is the knee ($\beta = 5$ changes nothing). The latency reduction depends on the execution model exactly as the static analysis of §10.5 predicts. Under the concurrent model—where specialists sharing a loaded model run in one batched pass, so the specialist stage costs a single call—the reduction is pinned at 9.1–9.3% regardless of k , because latency then depends on *whether* specialists run, not how many. Under the sequential model the reduction is 27–35% and grows with the pruning. We report the concurrent figure as the conservative headline and the sequential figure as the serial-execution case, consistent with the dual treatment in §10.5.

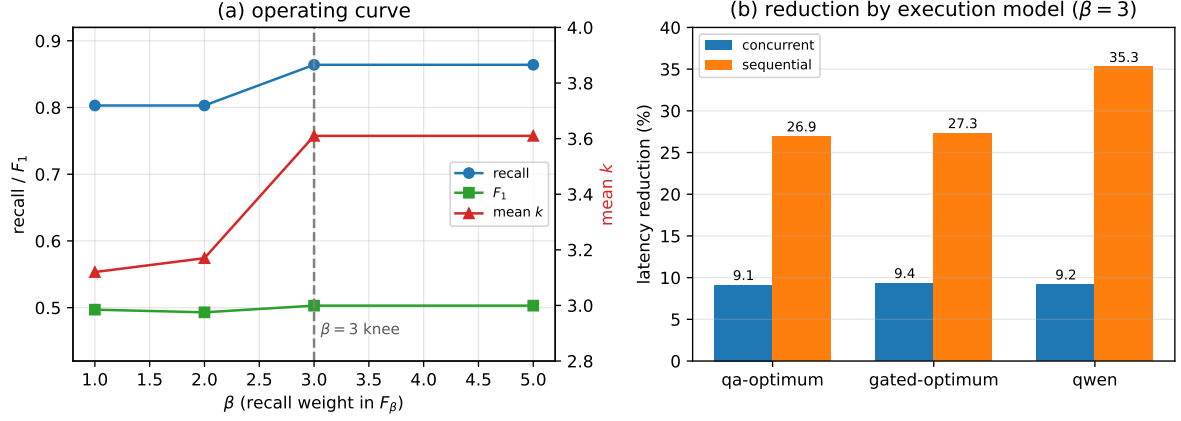


Figure 34: (a) Quality versus the F_β objective: recall and k rise with β , F_1 is flat, $\beta = 3$ is the knee. (b) Latency reduction is $\approx 9\%$ under concurrent serving (insensitive to k) and 27–35% under sequential serving (sensitive to k)—the same execution-model dichotomy as §10.5.

Calibration scale is the dominant factor. Table 6 and Fig. 35 isolate the effect of calibration size, and it echoes the small- n caveat that runs through §8. With only 25 routing-annotated queries over nine domains (2–3 positives each), the per-domain F_β fit overfits and produces degenerate thresholds up to 0.467; the gate becomes precision-obsessed (0.92 precision, 0.39 recall, $k = 0.6$), abstaining on almost everything, at $F_1 = 0.37$. Scaling to the 94-query calibration set (7–29 positives per domain) lifts F_1 to 0.50, restores recall to 0.86, and caps the maximum threshold at 0.166. The robustness controls of §12.1 are the safety net for the thin-sample regime; with enough calibration they become largely non-binding—sweeping the recall floor over $\{0.7, 0.8, 0.9, 1.0\}$ leaves F_1 within $[0.48, 0.50]$, so the operating point is governed by the data and the objective, not by a finely-tuned threshold.

Table 6: Calibration-scale ablation ($\beta = 3$, evaluated on the same 64-query test set). A larger calibration set removes the degenerate thresholds of the small-sample regime and rebalances the gate from near-total abstention to a productive precision/recall trade.

calibration	F_1	prec.	rec.	k	max τ
25-query	0.37	0.92	0.39	0.62	0.467
94-query	0.50	0.43	0.86	3.61	0.166

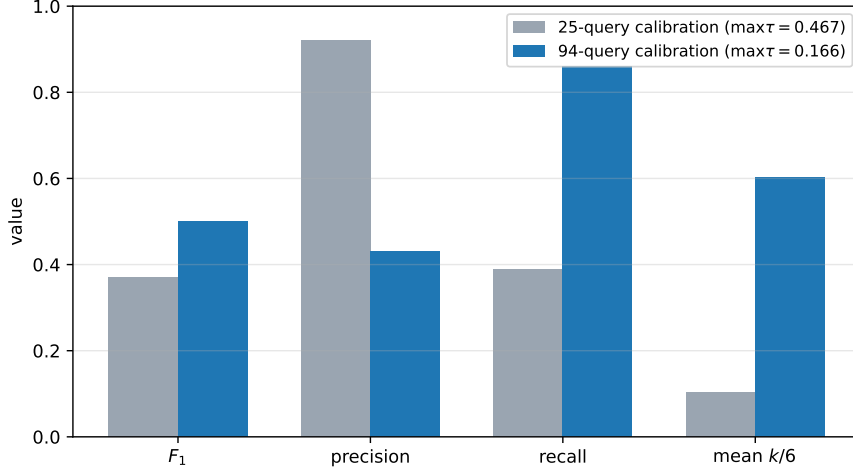


Figure 35: Scaling calibration from 25 to 94 queries raises routing F_1 from 0.37 to 0.50 and eliminates the degenerate, over-abstaining thresholds of the small-sample regime (mean k shown divided by 6).

12.3 Out-of-distribution behaviour

A legal assistant must decline questions outside its competence. On the 15 out-of-scope queries of the test set the LLM-free gate abstains on 0.60 (9/15), with mean false activation 0.80. The structure of the failures is the finding (Fig. 36): every leak is a succession- or family-adjacent query that is *on-topic but out-of-scope*—foreign-jurisdiction questions (Brazilian succession, German custody, Spanish property) and malformed inputs (a bare keyword, a wrong-language query, lorem-ipsum noise)—each routed into the topically-nearest Italian domain. A purely semantic gate cannot separate “Italian succession” from “Brazilian succession,” which are near-identical in embedding space: *the gate captures topic but not jurisdiction*. Detecting jurisdiction would require an explicit signal (a language or country classifier, or jurisdiction-tagged corpora), which we leave to future work. Under the deployed LLM dispatcher (rather than the LLM-free retrieval router evaluated here) abstention is substantially higher; the two configurations answer different questions and we report them separately rather than interchangeably.

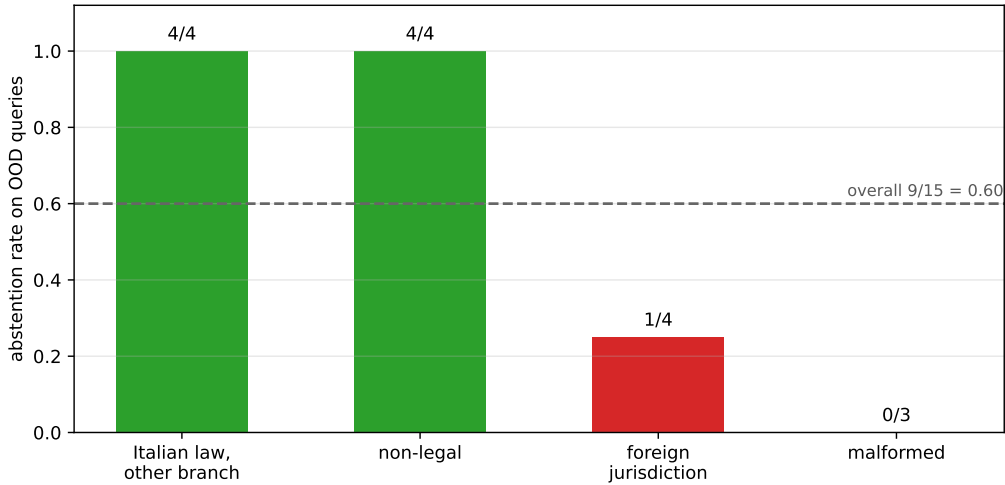


Figure 36: OOD abstention. The gate abstains on every topically-unrelated query and leaks only on topic-near, scope-far inputs (foreign jurisdiction, malformed), isolating a semantic-gating limitation rather than a calibration failure.

12.4 A single-objective concurrent-batch allocator

The dynamic layer is orthogonal to the static allocation; we close by strengthening the static model itself for the concurrent regime that §10.5 identified as the realistic on-device execution mode. We pose a compact single-objective allocator that maximizes pipeline quality $Q = F_d + \frac{1}{|D|} \sum_{\delta} Q_{s,\delta} + Q_y$ under a hard memory budget and a hard latency SLA T° , using the exact concurrent latency of §10.5,

$$L_{\text{total}} = L_d(c_d) + \Lambda + L_y(c_y), \quad \Lambda \geq \ell_c^g x_{s,c} \quad \forall c, \quad (28)$$

where the single auxiliary Λ collapses to the chosen specialist’s latency (exactly one $x_{s,c} = 1$), so a batched specialist stage costs one call. Solved with HiGHS on the measured Qwen instance at $M = 6.99$ GB and $T^\circ = 8$ s, it selects a **Qwen2.5-3B (Q8)** dispatcher and a *single shared* **Qwen2.5-1.5B (Q5)** group serving both specialist and synthesiser, at $Q = 1.88$, $L = 8.00$ s (the SLA binds), 5.43 GB: only two weight groups are loaded, and it is precisely the load-once sharing of the 1.5B group that frees the memory for the 8-bit 3B dispatcher. It beats the naive baselines (largest-fits 1.39, uniform 1.60); the per-role-best baseline reaches a higher nominal $Q = 1.95$ but is *infeasible*, breaching the SLA at 11.0s while fitting in memory—the joint memory–latency coupling the MILP respects and the heuristic ignores. The sequential-versus-concurrent comparison (Fig. 37) restates, in this single-objective setting, the result of §10.5: concurrent serving attains $Q \geq$ sequential at *every* SLA and remains feasible at tight SLAs ($T^\circ = 4$ –6 s) where the sequential model admits no feasible allocation. Concurrent batching both enlarges the feasible region and dominates on quality, independently confirming the execution-mode robustness reported for the full pool in §10.5.

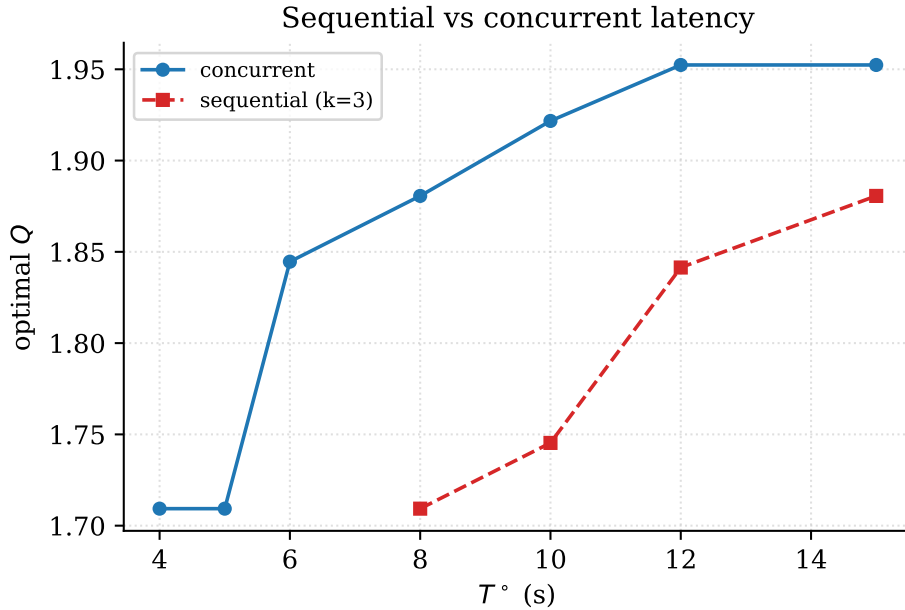


Figure 37: Single-objective concurrent-batch allocator: concurrent ($L_d + \Lambda + L_y$) versus sequential ($L_d + kL_s + L_y$) serving on the measured Qwen instance. Concurrent serving attains higher quality at every SLA and is feasible in the shaded region where sequential serving is not, matching the full-pool finding of §10.5.

12.5 What the dynamic layer establishes

Holding a measured static allocation fixed, a calibrated LLM-free gate that selects specialists per query improves routing F_1 by 0.17 and reduces latency by 10–33% (concurrent vs. sequential),

with a gain independent of the underlying allocation and an operating point set by calibration data rather than hand tuning. The improvement is purchased at no additional memory—the binding resource of the whole paper—because the gate is a function of an embedding similarity, not a second model. Two honest bounds carry into §13: the in-domain metric is routing accuracy, not reference-checked answer correctness, and the LLM-free gate’s OOD abstention is limited by its inability to detect jurisdiction. The complementary single-objective concurrent allocator reproduces, in a compact form, the execution-mode result of §10.5: batched serving Pareto-dominates sequential serving in quality and feasibility.

13 Discussion and Limitations

What the optimization establishes—and what it does not. The objective is distinct-model parameter count, a transparent *capability proxy*, not measured answer quality. The frontier states *how much distinct model* the pipeline can afford at each latency, not *how good* its answers are. The answer-quality measurement (judge-free embedding-based scoring of routing, grounding, and synthesis) is still running, and we fabricate no quality numbers. The same MILP accepts a quality objective directly—replace π_m in (11) with measured per-role quality and gate the specialist term by routing accuracy—so the present result is a coherent performance-space allocation and a validated scaffold for the quality-aware version. Because the measured-quality campaign (§8) shows capability does not track parameter count, we expect the quality-optimal frontier to differ from Fig. 7, and do not present the latter as a quality result.

An honest assessment of the optimization’s value. On the performance-only proxy a greedy sort matches the MILP on 92% of the frontier (§7): for that sortable objective we do not claim the optimization is necessary. Its value appears when the objective stops being sortable. We demonstrate this with heterogeneous specialists—distinct legal domains with per-domain model affinities—where the MILP beats greedy on 92% of points and on *all* points once enough specialists are active, because the best joint assignment trades model quality across domains under shared budgets in a way a per-slot sort cannot. That demonstration uses *synthetic* affinities and is labelled as such; its role is to locate the regime where optimization is required. The production objective—measured per-domain answer quality—lives in exactly that non-sortable regime, which is why the MILP, not a heuristic, is the right tool once the quality table exists.

Other limitations. (1) The serial-sum latency (7) assumes the activated specialists share one loaded instance and run one after another (the conservative worst case); if they are instead batched into a single forward pass the stage cost falls to one generation, which we model exactly in §10.5 and find the optimum unchanged. True latency on a given runtime lies between these two extremes. (2) All measurements are from a single device under one driver; absolute numbers shift on other hardware, though the qualitative curves should transfer and the perf table is cheap to re-measure. (3) Derived latency (6) assumes throughput independent of output length, mildly optimistic for long outputs. (4) Peak VRAM via NVML may under-report transient spikes. (5) The activation count k is treated as a parameter, not modelled from the query distribution; the $k=3$ curve approximates the dataset mean of 2.68. (6) The quality evaluation rests on 37 root queries and is under-powered per domain (7 of 9 domains have $n < 10$); we report bootstrap CIs and rely on pooled and cross-role signals rather than individual small- n per-domain claims. A leave-one-domain-out check (§8.4) indicates the specialist selection is not in-sample overfitting—the train-selected model is Llama-3.2-1B in all nine folds with a 0.001 mean held-out generalization gap—but with only 37 root queries the quality-aware frontier is still best read as an in-sample optimum whose absolute values are optimistic, even though the *selection* generalizes. (7) The primary quality-aware objective (18) is additive; we relax this with a measured bilinear router–specialist gating (§10), but that estimate uses

the same small per-domain samples, so its specific dispatcher winner is indicative rather than definitive. (8) RAGAS embedding-based scores are taken as the quality ground truth and not validated against human legal judgement; the synthesiser aggregate in particular reduces largely to correctness. (9) The batching analysis (§10.5) *parametrizes* the sequential→batched range via the serialization factor σ ; the performance table is measured single-stream, so we do not claim a specific measured batching speedup—we show only that the optimal allocation is invariant across the whole σ range. Measuring batched throughput directly is left for future work. (10) The non-monotone three-regime finding and the value of modelling router–specialist coupling are both established on a heterogeneous pool; §11 shows each collapses within a single dense family (where, depending on the family, the size–quality relation is monotone in either direction, and the gated and additive optima coincide). The external validity of both results is therefore bounded to multi-family catalogues—which are the realistic deployment setting, but the single-family case should not be expected to exhibit them. (11) The dynamic-layer evaluation (§12) scores *routing* accuracy against gold `expected_agents`—a proxy for, not a measurement of, answer correctness; we have no reference answers for the 64-query test set, so the dynamic layer’s in-domain numbers establish that it activates the right specialists, not that the resulting answers are better, though removing distractor specialists makes a quality loss unlikely. (12) The LLM-free gate’s out-of-distribution abstention (0.60) is bounded by its inability to detect *jurisdiction*: it captures topic but routes foreign-jurisdiction queries into the topically-nearest Italian domain (§12.3). The higher abstention obtainable with the deployed LLM dispatcher is a different configuration and is not interchangeable with the LLM-free figure; we report them separately.

14 Conclusion

We characterized the performance of 75 quantized SLM configurations on a single 8 GB GPU—throughput, memory, latency, energy, their precision/context/scale cost curves—and we posed and solved an explicit MILP that allocates models to the pipeline’s three generation roles to maximize distinct-model capacity under a load-once memory budget and a serial system-latency budget, parametric in the number of activated specialists. We held the optimization to a strict standard: for the sortable parameter objective a greedy heuristic matches it on 92% of points, which we report rather than hide. The optimization becomes necessary, not marginal, once the objective is not sortable—demonstrated (on synthetic per-domain affinities) by heterogeneous specialists, where the MILP wins on 92% of points and on all points when enough specialists are active. Finally, the measured-quality campaign across all three roles reveals three distinct scaling laws: routing (dispatcher) and synthesis quality rise with model size, while domain-grounded specialist quality is non-monotone and peaks at a 1.2B model—so the roles prefer opposite ends of the size spectrum, the capacity proxy is sub-optimal in quality at every latency budget, and per-role allocation is empirically justified. Acting on this, the quality-aware MILP—the same program with measured quality replacing the proxy—traces the measured quality–latency frontier, independently recovers the mixed three-regime optimum (large dispatcher, small specialist, large synthesiser), beats fixed-model policies, and dominates the parameter-proxy allocator at every matched-latency budget by a mean of 0.13 quality points. Modelling the router–specialist coupling as a bilinear term—exactly linearized and solved on measured data—goes further: the gated optimum dominates the coupling-blind additive optimum at 42 of 44 budgets and reveals that the right router is the one with high recall *on the activated domains*, not high global F1. We give a mechanistic account of the three regimes—a role’s optimal model size tracks its grounding requirement, so classification-like roles (routing) reward scale while context-faithful roles (grounded specialists) penalize it—turning the empirical finding into a testable prediction. And we characterize *when* optimization is worth it: the MILP-over-greedy gap grows near-linearly with how strongly the pipeline gates downstream

agents on upstream decisions, from 0.06 (decoupled) to 0.18 (full measured gating). Finally, a generality analysis isolates what the conclusion depends on: both the non-monotone specialist regime and the value of modelling router–specialist coupling are cross-family phenomena that collapse within a single dense family, whereas the small-specialist optimum is robust to the serial-versus-batched execution model—so the result is driven by pool diversity rather than by the execution accounting. The central lesson is that a parameter proxy and a measured-quality objective induce materially different optimal configurations, and that role couplings matter: in a multi-agent pipeline, allocation must be driven by measured, coupling-aware per-role quality, which a single global heuristic cannot replace. Finally, the allocated substrate can itself be made query-adaptive at no memory cost: a calibrated, LLM-free per-query gate that selects which specialists to activate raises routing F_1 from 0.33 to 0.50 and cuts latency 10–33% with a gain independent of the underlying allocation, and a single-objective concurrent-batch MILP confirms batched serving Pareto-dominates sequential serving in quality and feasibility—so static allocation, coupling-aware optimization, and per-query adaptation compose into one coherent on-device design. All data, the MILP, the baselines, and the plotting code are released.

Reproducibility. The performance table (75 configurations), the MILP (`syscap_milp.py`, PuLP/CBC), the frontier data (`syscap_frontier.csv`), and all plotting scripts are released. Latency and energy are derived via Eq. (6) at $T = 384$ tokens. Capacity is parameter count, an explicit capability proxy, not measured quality. The headline capacity–latency frontiers (Fig. 7, Table 3) use the serial chain of Eq. (7) and are reproduced by the released solver with `--latency-model sequential --k-activated k` for $k \in \{1, 3, 5, 9\}$; the exact batched model of §10.5 is the solver’s `worst_case` mode, and the interpolating σ -sweep is also released. The dynamic-layer experiments of §12 are likewise released: the per-query routing/latency evaluation (`routing_eval.py`), the out-of-distribution abstention test (`ood_eval.py`), the end-to-end pipeline runner (`run_pipeline.py`), and the single-objective concurrent-batch allocator (HiGHS), together with the three disjoint query sets used—the 25-query gold-passage profiles, the 94-query routing-annotated calibration set, and the 64-query held-out test set (§12). The gate’s thresholds are fit by the per-domain F_β procedure of Eq. (27) with $\beta = 3$. The §12 routing/abstention numbers use `bge-m3` relevance scores; the released package replays them offline once the one-time score cache (`build_score_cache --embedder bge-m3`) is committed, and a dependency-free hashing embedder validates the pipeline end-to-end without it.

References

- [1] Y. Jiang, F. Fu, W. Zhao, S. Rabanser, N. D. Lane, and B. Yuan, “Cascadia: An Efficient Cascade Serving System for Large Language Models,” *arXiv:2506.04203*, 2025.
- [2] (Cascade-Aware Training of Language Models), *arXiv:2406.00060*, 2024.
- [3] “Semantic Agreement Enables Efficient Open-Ended LLM Cascades,” *arXiv:2509.21837*, 2025.
- [4] J. Dekoninck, M. Baader, and M. Vechev, “A Unified Approach to Routing and Cascading for LLMs,” *ICML*, 2025 (arXiv:2410.10347).
- [5] “Dynamic Model Routing and Cascading for Efficient LLM Inference: A Survey,” *arXiv:2603.04445*, 2026.
- [6] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, “RouteLLM: Learning to Route LLMs with Preference Data,” *arXiv:2406.18665*, 2024.
- [7] J. Wang, J. Wang, B. Athiwaratkun, C. Zhang, and J. Zou, “Mixture-of-Agents Enhances Large Language Model Capabilities,” *arXiv:2406.04692*, 2024.
- [8] “Multi-Agent Collaboration Mechanisms: A Survey of LLMs,” *arXiv:2501.06322*, 2025.

- [9] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “RAGAs: Automated Evaluation of Retrieval Augmented Generation,” *EACL System Demonstrations*, 2024 (arXiv:2309.15217).
- [10] “SLMQuant: Benchmarking Small Language Model Quantization for Practical Deployment,” *arXiv:2511.13023*, 2025.
- [11] “Edge Deployment of Small Language Models: A Survey,” *arXiv:2511.22334*, 2025.
- [12] “On-Device LLMs: State of the Union, 2026,” technical report, 2026.
- [13] “Which Quantization Should I Use? A Unified Evaluation of `llama.cpp` Quantization on Llama-3.1-8B-Instruct,” *arXiv:2601.14277*, 2026.